

QUIZ

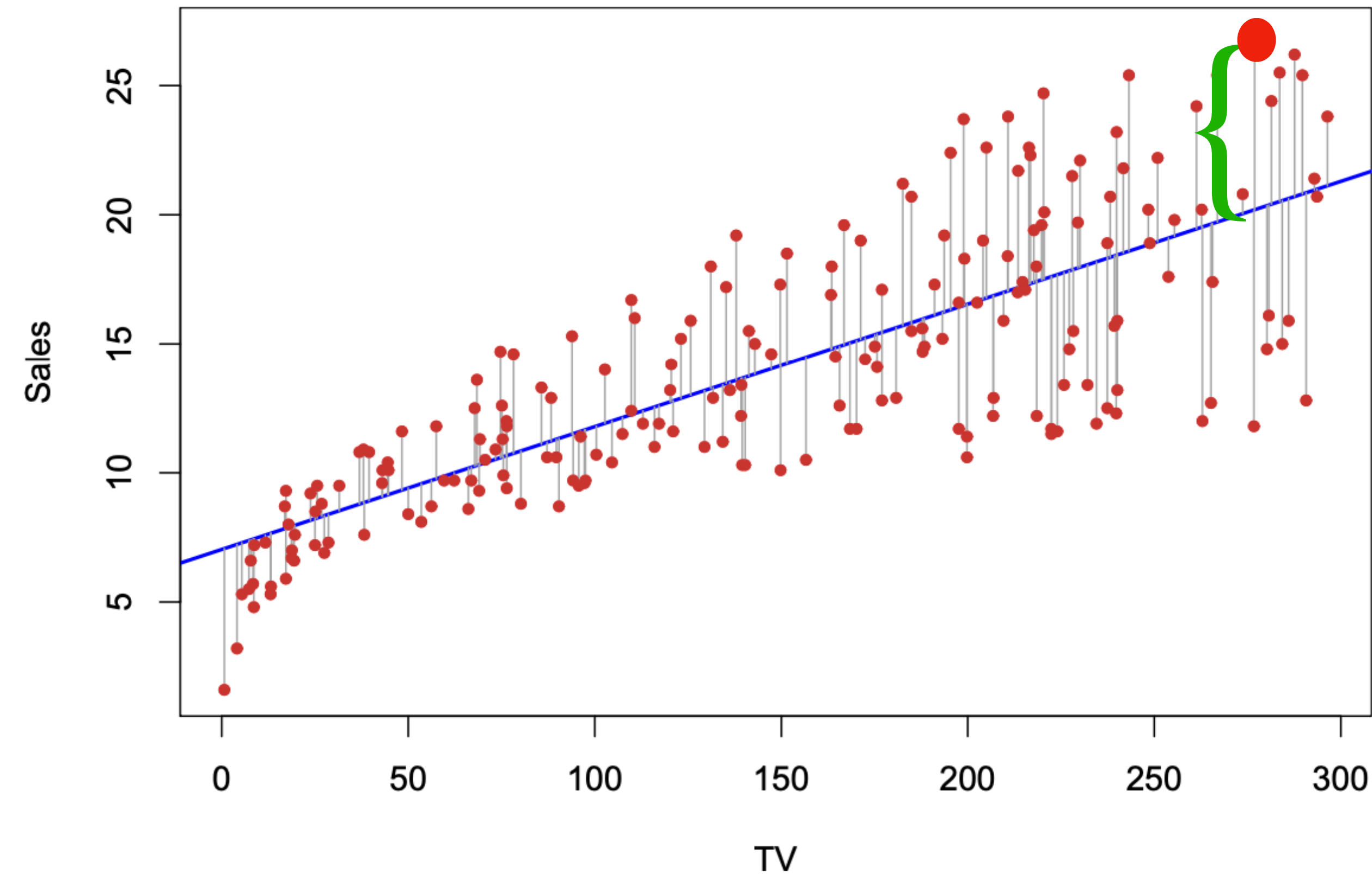
What is the objective function to minimize in linear regression?

REVIEW OF THE PREVIOUS LECTURE

Simple Linear Regression

‘Best’ fitting $\hat{y} = \hat{w}_0 + \hat{w}_1 x$:

Minimize $RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$



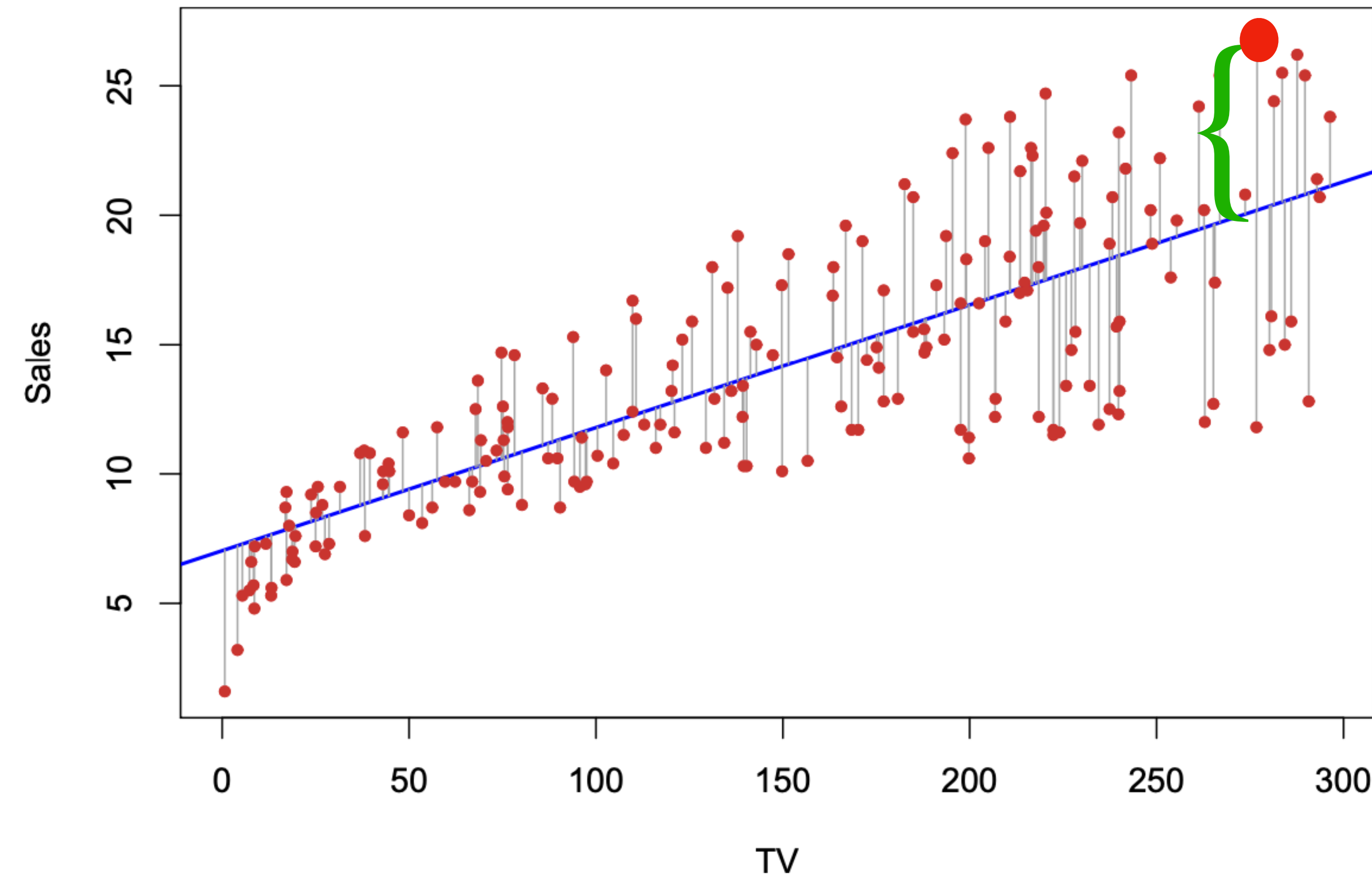
REVIEW OF THE PREVIOUS LECTURE

Simple Linear Regression

‘Best’ fitting $\hat{y} = \hat{w}_0 + \hat{w}_1 x$:

Minimize $RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$

Accuracy of the model RSE, R^2



REVIEW OF THE PREVIOUS LECTURE

Simple Linear Regression

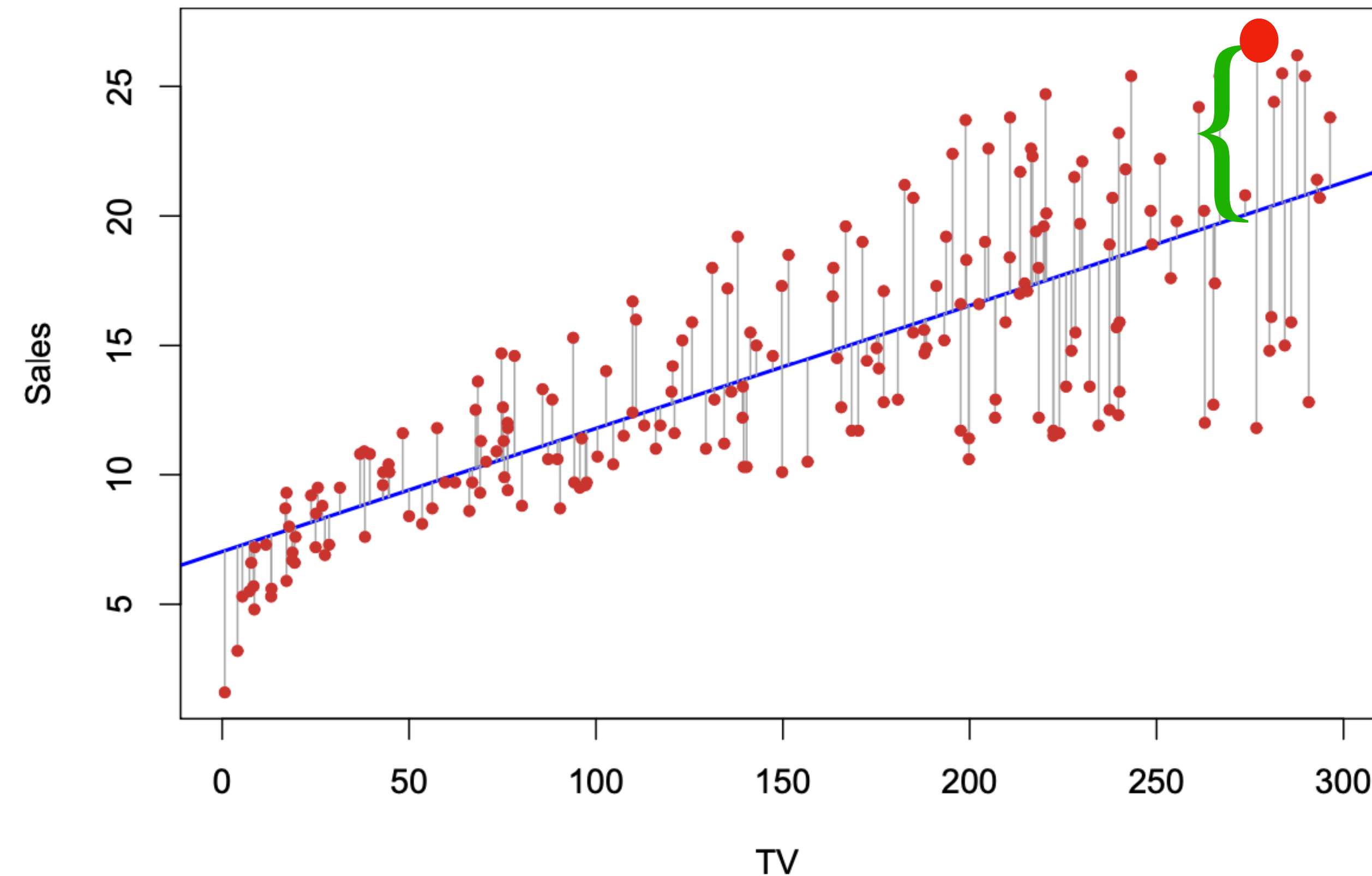
‘Best’ fitting $\hat{y} = \hat{w}_0 + \hat{w}_1 x$:

Minimize $RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$

Accuracy of the model RSE, R^2

Multiple Linear Regression

$$\hat{y} = \hat{w}_0 + \hat{w}_1 x_1 + \hat{w}_2 x_2 + \cdots \hat{w}_m x_m$$



OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Synergy/Interaction effect:

$$\widehat{sales} = \hat{w}_0 + \hat{w}_1 TV \quad R^2 \approx 0.58$$

$$\widehat{sales} = \hat{w}_0 + \hat{w}_1 TV + \hat{w}_2 radio \quad R^2 \approx 0.90$$

One unit increase in TV increases $sales$ by $\hat{w}_1$ units.

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Synergy/Interaction effect:

$$\widehat{sales} = \hat{w}_0 + \hat{w}_1 TV \quad R^2 \approx 0.58$$

$$\widehat{sales} = \hat{w}_0 + \hat{w}_1 TV + \hat{w}_2 radio \quad R^2 \approx 0.90$$

One unit increase in TV increases $sales$ by $\hat{w}_1$ units.

What if more *radio* ads increase effectiveness of *TV* advertising?

Splitting \$100 between TV , $radio$ may be better than all on TV or all on $radio$.

Unit increase in TV affects $sales$ in amount dependent also on $radio$.

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Synergy/Interaction effect:

How to achieve it?

Introduce a new interaction term as a product of the two.

$$\widehat{sales} = \hat{w}_0 + \hat{w}_1 \times TV + \hat{w}_2 \times radio + \hat{w}_3 \times TV \times radio$$

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Synergy/Interaction effect:

How to achieve it?

Introduce a new interaction term as a product of the two.

$$\widehat{sales} = \hat{w}_0 + \hat{w}_1 \times TV + \hat{w}_2 \times radio + \hat{w}_3 \times TV \times radio$$

$$= \hat{w}_0 + (\hat{w}_1 + \hat{w}_3 \times radio) \times TV + \hat{w}_2 \times radio$$

$$= \hat{w}_0 + \hat{w}'_1 \times TV + \hat{w}_2 \times radio$$

$$\text{where } \hat{w}'_1 = \hat{w}_1 + \hat{w}_3 \times radio$$

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Ex: Back to the Advertising data set.

```
# add new feature tv*radio to dataframe
df['TV_radio'] = df['TV'] * df['radio']
df.head()
```

	TV	radio	newspaper	sales	TV_radio
0	230.1	37.8	69.2	22.1	8697.78
1	44.5	39.3	45.1	10.4	1748.85
2	17.2	45.9	69.3	9.3	789.48
3	151.5	41.3	58.5	18.5	6256.95
4	180.8	10.8	58.4	12.9	1952.64

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Train with TV, Radio, TV_radio. Measure R^2 .

```
# train/test with TV, radio, and new feature|
X_new = df[['TV', 'radio', 'TV_radio']]
X_train_new, X_test_new, _, _ = \
    train_test_split(X_new, y, train_size=0.8, random_state=0)

# linear regression using TV, radio, TV*radio
lr_new = linear_model.LinearRegression()
lr_new.fit(X_train_new, y_train)
```

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Train with TV, Radio, TV_radio. Measure R^2 .

```
# train/test with TV, radio, and new feature|
X_new = df[['TV', 'radio', 'TV_radio']]
X_train_new, X_test_new, _, _ = \
    train_test_split(X_new, y, train_size=0.8, random_state=0)

# linear regression using TV, radio, TV*radio
lr_new = linear_model.LinearRegression()
lr_new.fit(X_train_new, y_train)
```

```
w_0: 6.652712826674431
w_1, w_2, w_3: [0.0194162  0.03790871 0.0010505 ]
R-squared: 0.9732546974049714
```

Considerable
improvement!

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Non-linear Relationships:

True relationship between response and predictors may be non-linear.

How to extend? **Polynomial regression.**

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

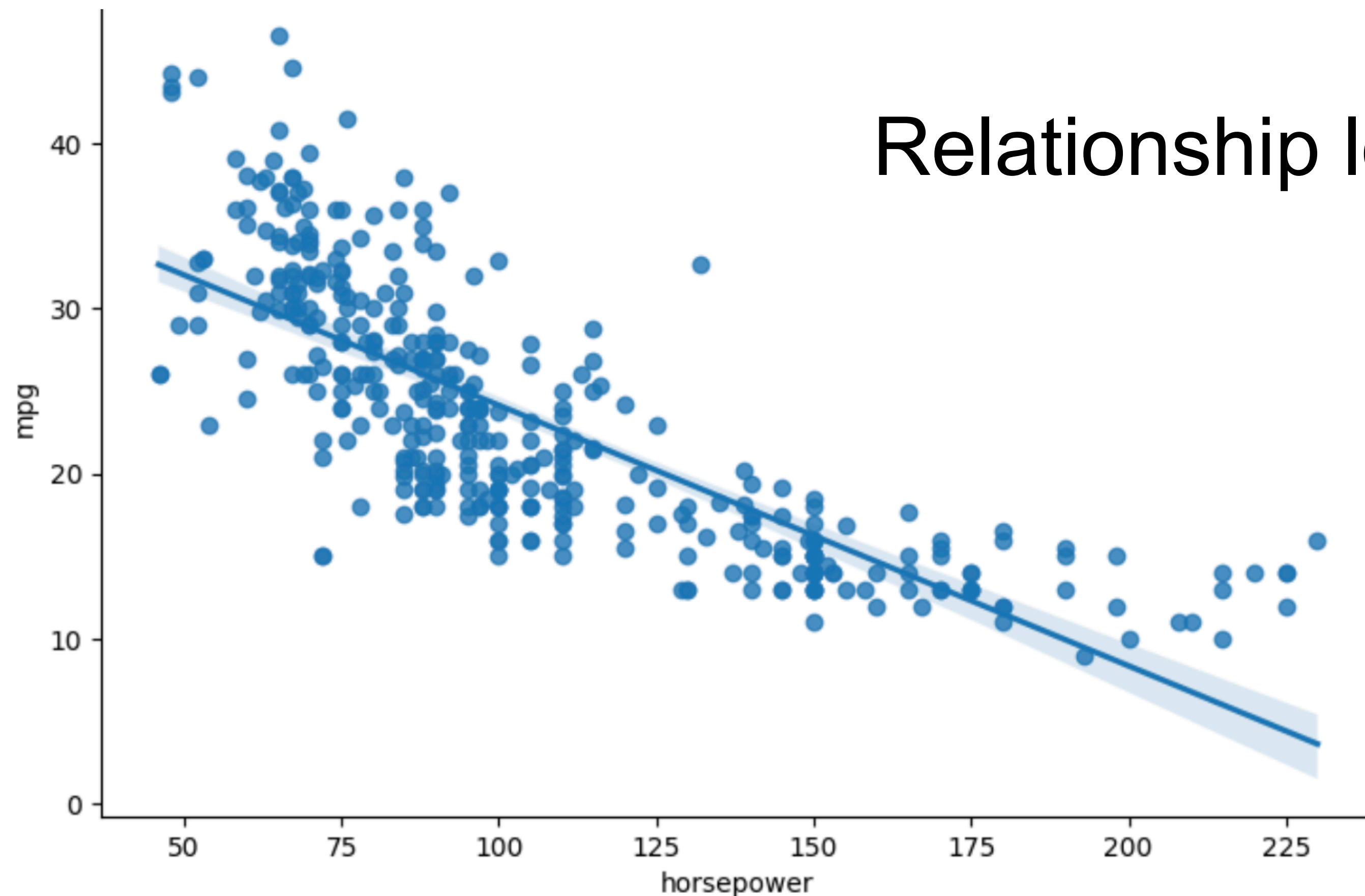
Ex: Auto data set.

	mpg	cylinders	displacement	horsepower	weight	acceleration	year	origin	name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino

Want to predict *mpg* from *horsepower*.

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

```
sns.pairplot(df, x_vars=['horsepower'], y_vars='mpg', height=5,  
              aspect =1.5, kind='reg')
```



Relationship looks quadratic rather than linear

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Train with *horsepower* only.

```
X = df[['horsepower']]
y = df.mpg
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, train_size=0.8, random_state=0)
# linear regression using TV, radio
lr = linear_model.LinearRegression()
lr.fit(X_train, y_train)
```

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Train with *horsepower* only.

```
X = df[['horsepower']]
y = df.mpg
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, train_size=0.8, random_state=0)
# linear regression using TV, radio
lr = linear_model.LinearRegression()
lr.fit(X_train, y_train)
```

```
w_0: 39.96223257893908
w_1: [-0.15844506]
R-squared: 0.5955625946891978
```

Doesn't look bad, but depends on the application area.

Possible improvement based on our previous observation?

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Add new feature $horsepower^2$ and train using the two.

```
# add new feature horsepower^2
df['horsepower2'] = df['horsepower']**2
# train/test with new feature, using same rows
X_2 = df[['horsepower', 'horsepower2']]
X_train_2, X_test_2, _, _ = \
    train_test_split(X_2, y, train_size=0.8, random_state=0)

# linear regression using TV, radio, TV*radio
lr_2 = linear_model.LinearRegression()
lr_2.fit(X_train_2, y_train)
```

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Add new feature $horsepower^2$ and train using the two.

```
# add new feature horsepower^2
df['horsepower2'] = df['horsepower']**2
# train/test with new feature, using same rows
X_2 = df[['horsepower', 'horsepower2']]
X_train_2, X_test_2, _, _ = \
    train_test_split(X_2, y, train_size=0.8, random_state=0)

# linear regression using TV, radio, TV*radio
lr_2 = linear_model.LinearRegression()
lr_2.fit(X_train_2, y_train)
```

$$mpg = \hat{w}_0 + \hat{w}_1 horsepower + \hat{w}_2 \times horsepower^2$$

(Polynomial regression)

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Add new feature horsepower^2 and train using the two.

```
# add new feature horsepower^2
df['horsepower2'] = df['horsepower']**2
# train/test with new feature, using same rows
X_2 = df[['horsepower', 'horsepower2']]
X_train_2, X_test_2, _, _ = \
    train_test_split(X_2, y, train_size=0.8, random_state=0)

# linear regression using TV, radio, TV*radio
lr_2 = linear_model.LinearRegression()
lr_2.fit(X_train_2, y_train)
```

```
w_0: 56.280426279112035
w_1, w_2: [-0.4546145  0.00118246]
R-squared: 0.673124476641667
```

Considerable improvement!

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Qualitative Predictors

So far all features in our linear regression were quantitative.

How to extend in case of qualitative features? **One hot encoding**

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

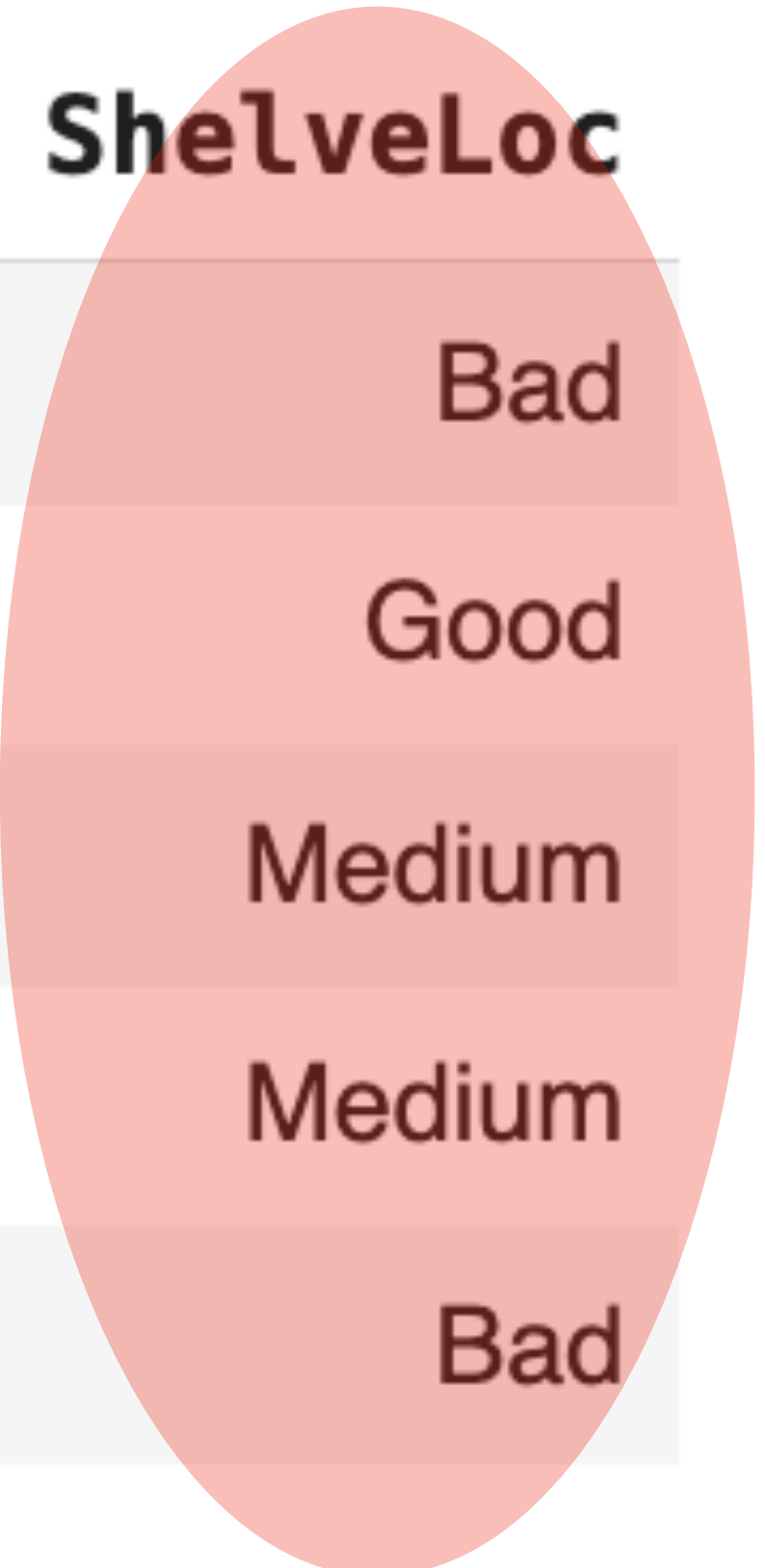
Ex: Carseats data set.

	Sales	CompPrice	Income	Advertising	Population	Price	ShelveLoc
0	9.50	138	73	11	276	120	Bad
1	11.22	111	48	16	260	83	Good
2	10.06	113	35	10	269	80	Medium
3	7.40	117	100	4	466	97	Medium
4	4.15	141	64	3	340	128	Bad

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Ex: Carseats data set.

	Sales	CompPrice	Income	Advertising	Population	Price	ShelveLoc
0	9.50	138	73	11	276	120	Bad
1	11.22	111	48	16	260	83	Good
2	10.06	113	35	10	269	80	Medium
3	7.40	117	100	4	466	97	Medium
4	4.15	141	64	3	340	128	Bad



Want to predict *Sales*. All features are numeric other than *ShelveLoc*.
How to use *ShelveLoc* in our prediction?

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Introduce dummy features as necessary.

```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(drop='first') # drop the first dummy variable
enc_df = pd.DataFrame(enc.fit_transform(df[['ShelveLoc']]).toarray())
enc_df.columns = ['Good', 'Medium']
df = df.join(enc_df)
df.head()
```

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Introduce dummy features as necessary.

```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(drop='first') # drop the first dummy variable
enc_df = pd.DataFrame(enc.fit_transform(df[['ShelveLoc']]).toarray())
enc_df.columns = ['Good', 'Medium']
df = df.join(enc_df)
df.head()
```

	Sales	CompPrice	Income	Advertising	Population	Price	ShelveLoc	Good	Medium
0	9.50	138	73	11	276	120	Bad	0.0	0.0
1	11.22	111	48	16	260	83	Good	1.0	0.0
2	10.06	113	35	10	269	80	Medium	0.0	1.0
3	7.40	117	100	4	466	97	Medium	0.0	1.0
4	4.15	141	64	3	340	128	Bad	0.0	0.0

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Introduce dummy features as necessary.

```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(drop='first') # drop the first dummy variable
enc_df = pd.DataFrame(enc.fit_transform(df[['ShelveLoc']]).toarray())
enc_df.columns = ['Good', 'Medium']
df = df.join(enc_df)
df.head()
```

Why just *Good, Medium*?

	Sales	CompPrice	Income	Advertising	Population	Price	ShelveLoc	Good	Medium
0	9.50	138	73	11	276	120	Bad	0.0	0.0
1	11.22	111	48	16	260	83	Good	1.0	0.0
2	10.06	113	35	10	269	80	Medium	0.0	1.0
3	7.40	117	100	4	466	97	Medium	0.0	1.0
4	4.15	141	64	3	340	128	Bad	0.0	0.0

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Introduce dummy features as necessary.

```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(drop='first') # drop the first dummy variable
enc_df = pd.DataFrame(enc.fit_transform(df[['ShelveLoc']]).toarray())
enc_df.columns = ['Good', 'Medium']
df = df.join(enc_df)
df.head()
```

Why just *Good, Medium*?

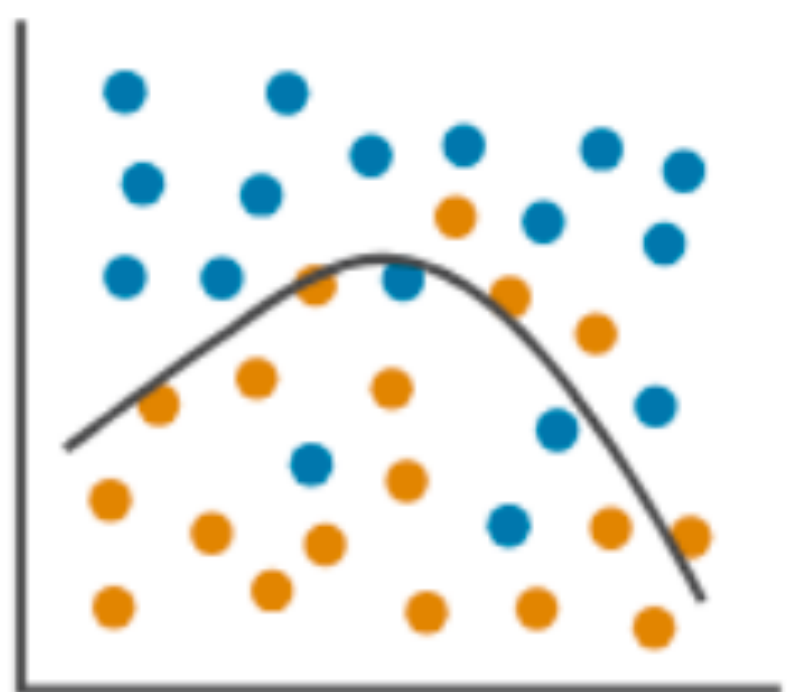
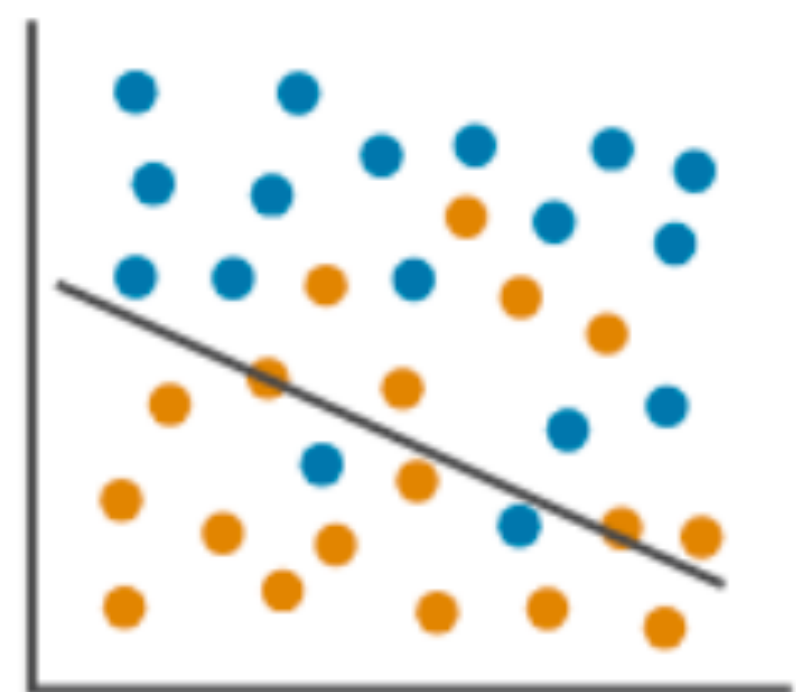
Intercept would be linear combination of dummy features.

	Sales	CompPrice	Income	Advertising	Population	Price	ShelveLoc	Good	Medium
0	9.50	138	73	11	276	120	Bad	0.0	0.0
1	11.22	111	48	16	260	83	Good	1.0	0.0
2	10.06	113	35	10	269	80	Medium	0.0	1.0
3	7.40	117	100	4	466	97	Medium	0.0	1.0
4	4.15	141	64	3	340	128	Bad	0.0	0.0

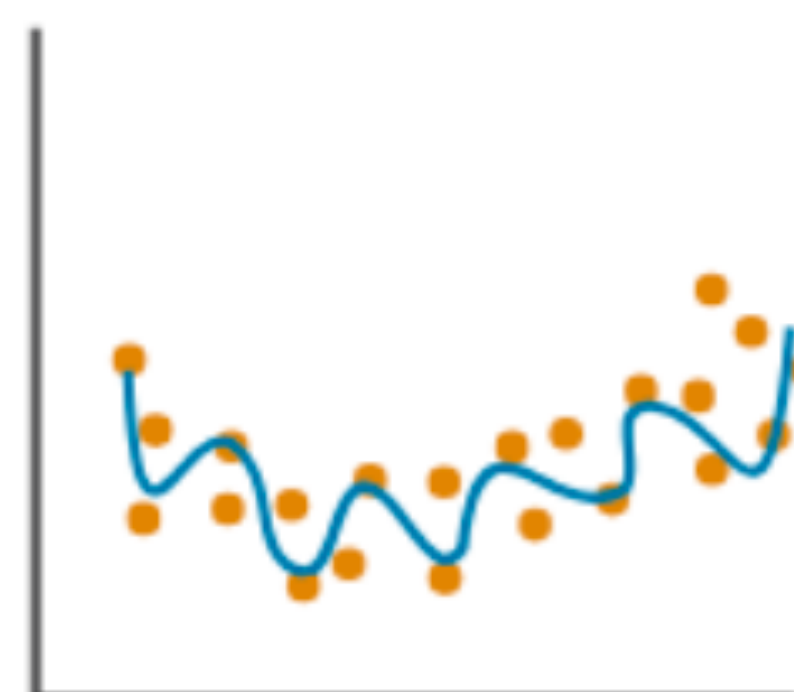
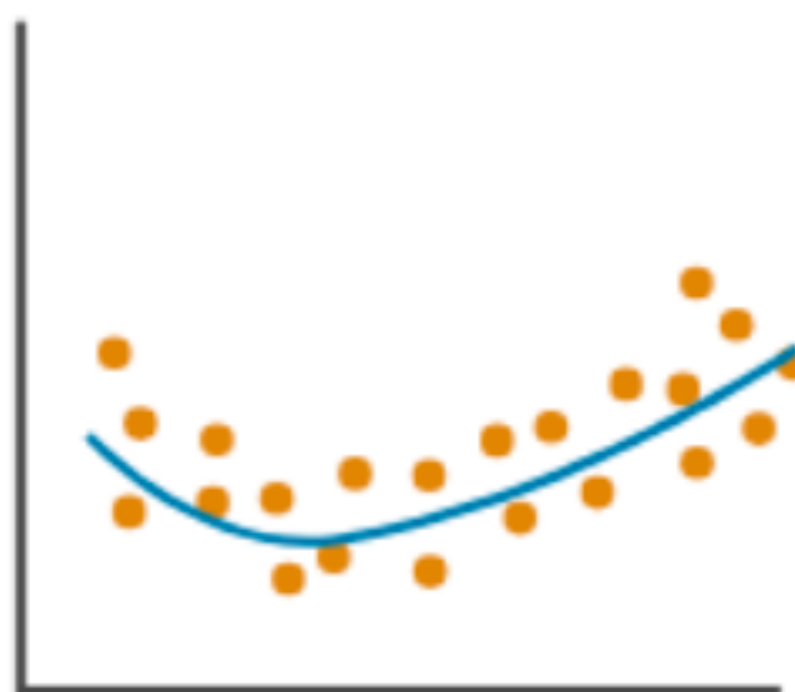
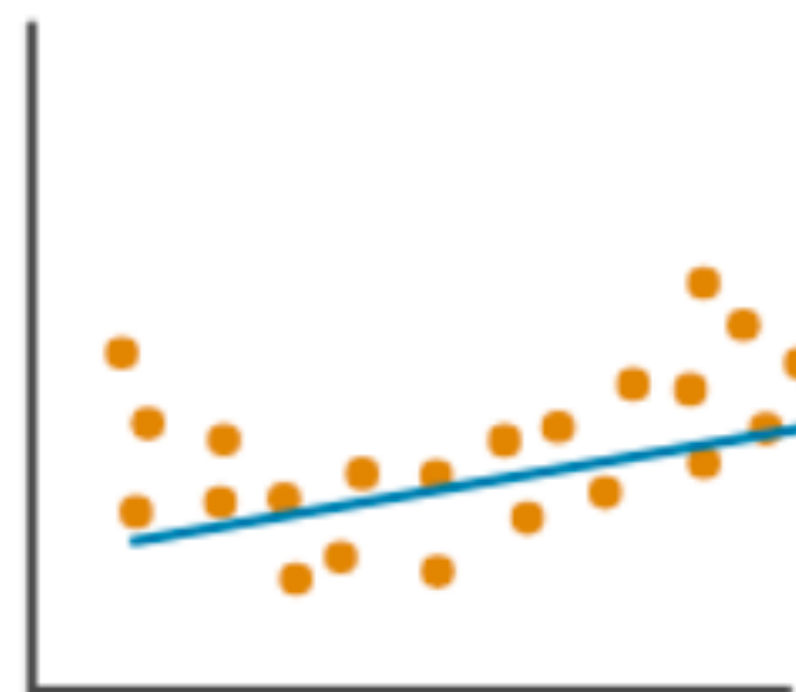
OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Restricting Model Complexity

Classification



Regression



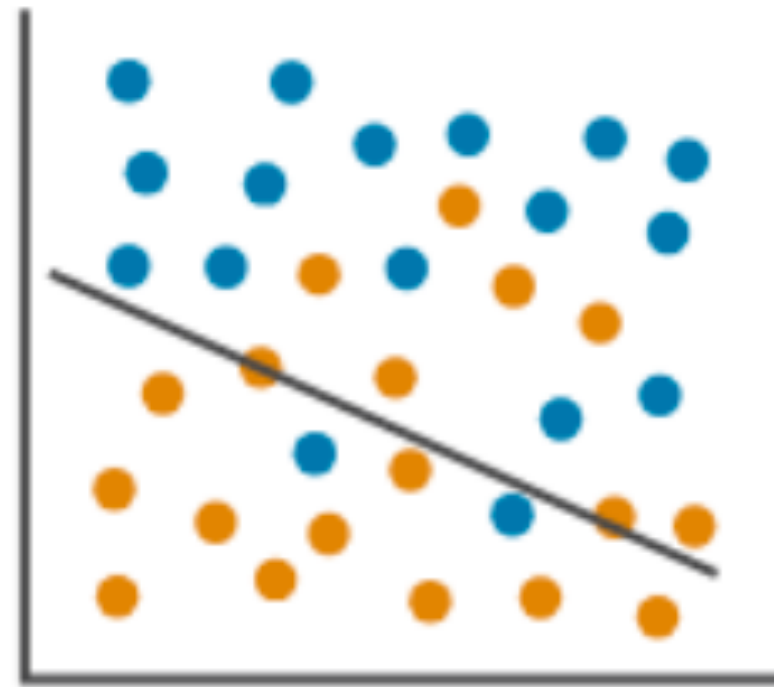
Which plots correspond to underfitting, right fit, and overfitting?

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

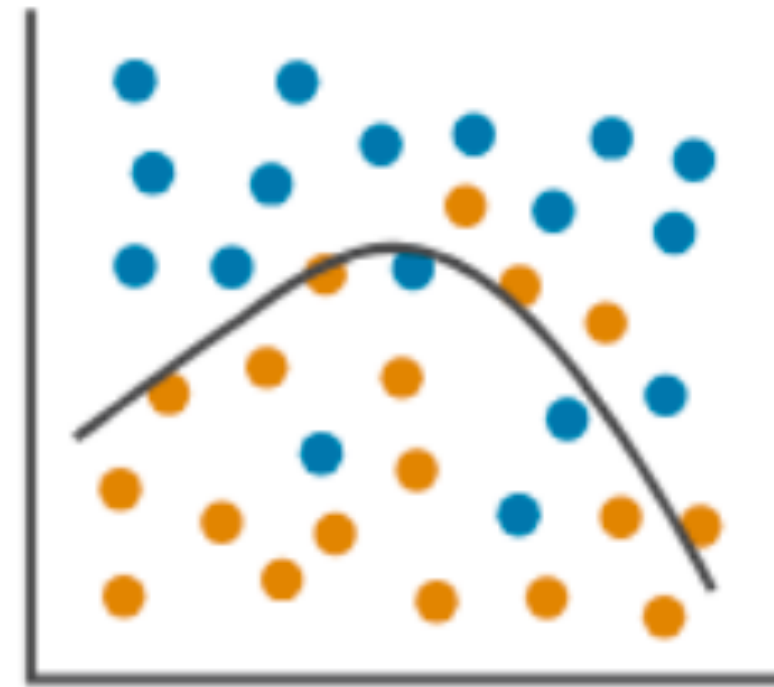
Restricting Model Complexity

Bias/variance tradeoff resulting in underfitting/overfitting.

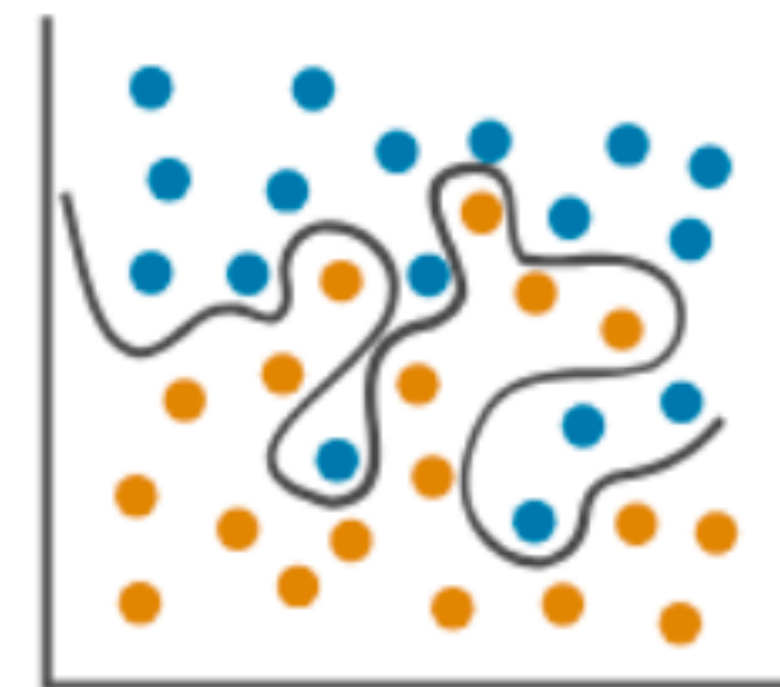
Classification



Underfit

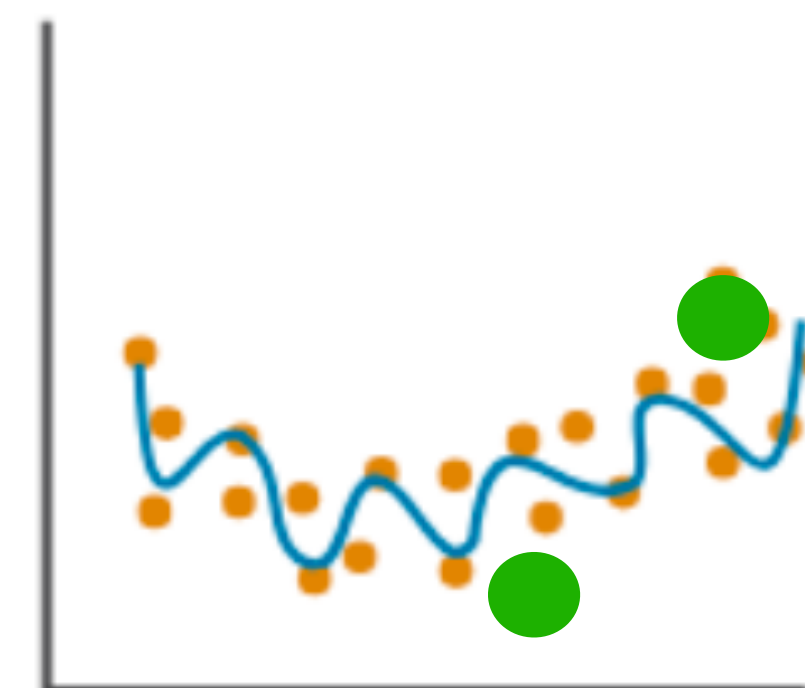
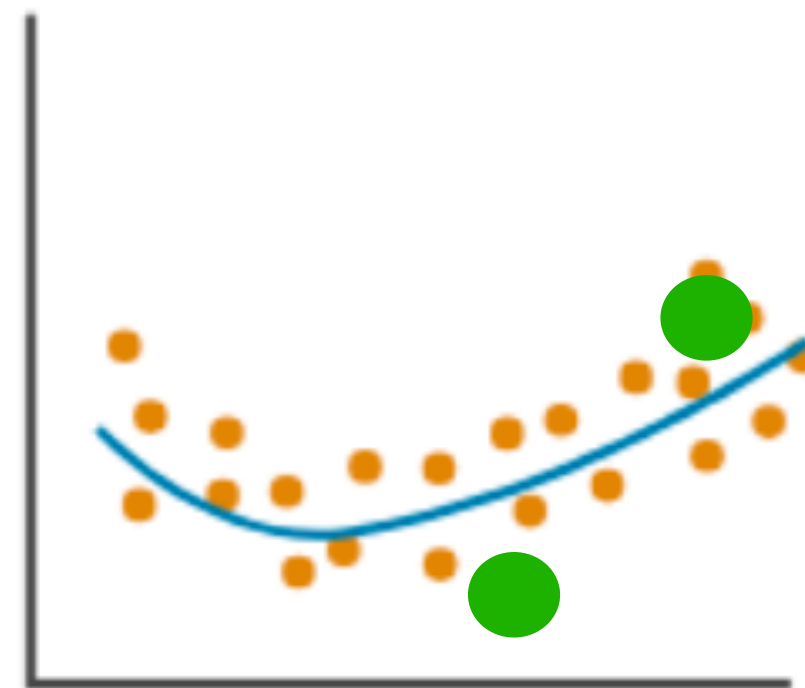
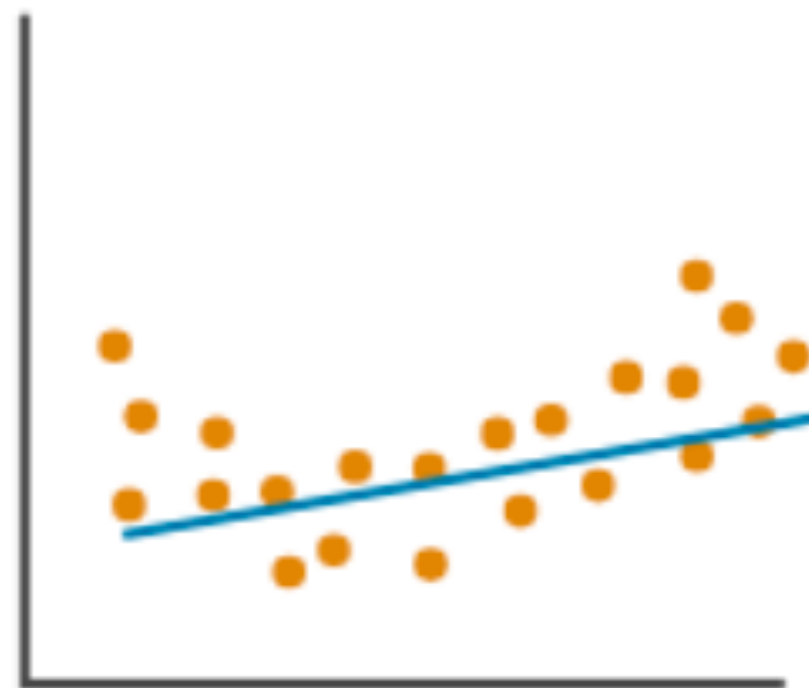


Right fit



Overfit

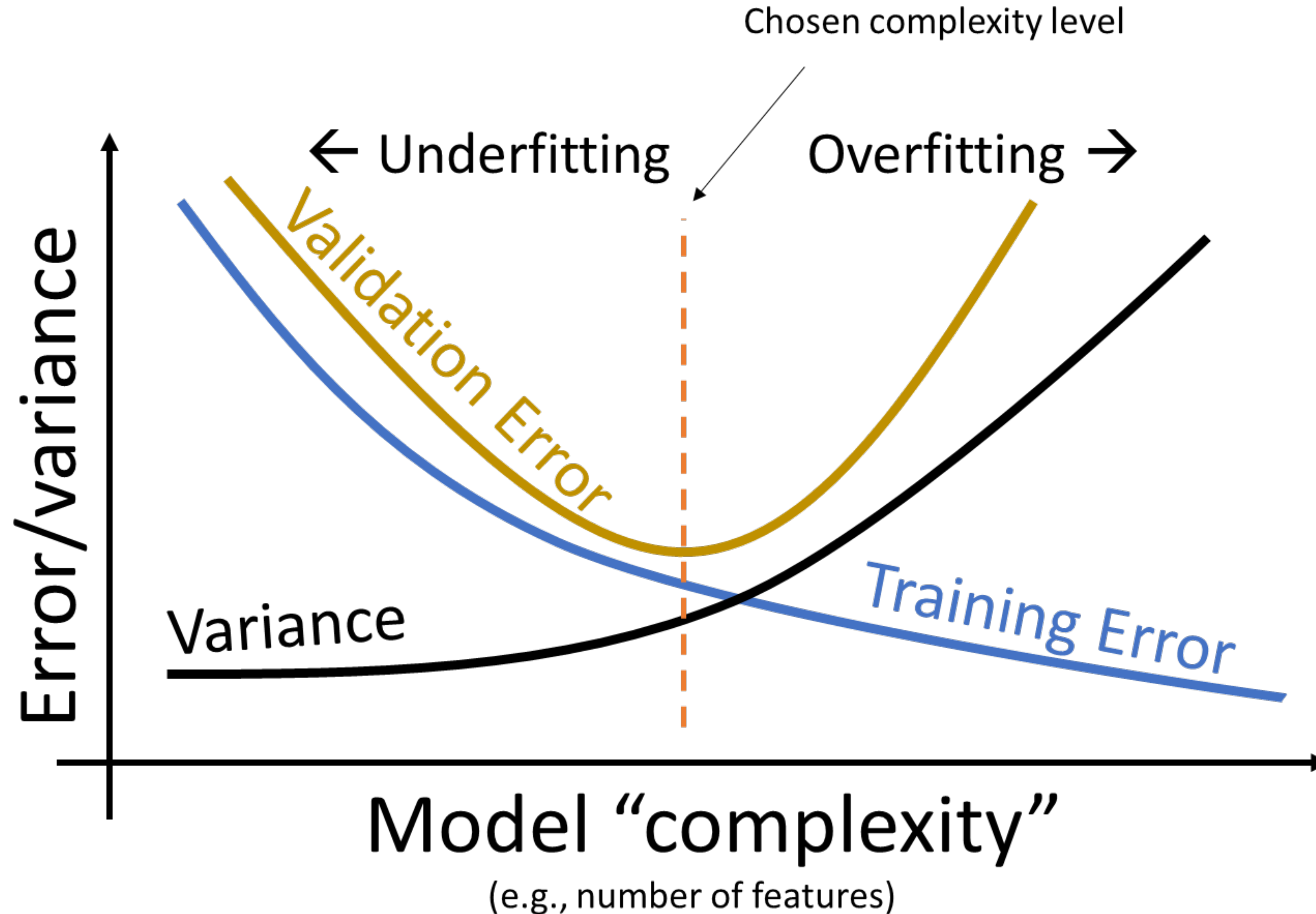
Regression



Consider green test points. Overfit model doesn't perform well!

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Restricting Model Complexity



RESTRICTING MODEL COMPLEXITY: REGULARIZATION

Loss function to minimize:

$$RSS = \sum_{i=1}^n (y_i - w_0^2 - (w_1x_{i1} + w_2x_{i2} + \cdots + w_mx_{im}))^2$$

RESTRICTING MODEL COMPLEXITY: REGULARIZATION

Loss function to minimize:

$$RSS = \sum_{i=1}^n (y_i - w_0 - (w_1x_{i1} + w_2x_{i2} + \cdots + w_mx_{im}))^2$$

To simplify, add constraints:

Alternative 1: $\sum_{i=1}^m |w_i| \leq t$ L1 Regularization (LASSO)

RESTRICTING MODEL COMPLEXITY: REGULARIZATION

Loss function to minimize:

$$RSS = \sum_{i=1}^n (y_i - w_0^2 - (w_1x_{i1} + w_2x_{i2} + \cdots + w_mx_{im}))^2$$

To simplify, add constraints:

Alternative 1: $\sum_{i=1}^m |w_i| \leq t$ L1 Regularization (LASSO)

Alternative 2: $\sum_{i=1}^m w_i^2 \leq t$ L2 Regularization (Ridge)

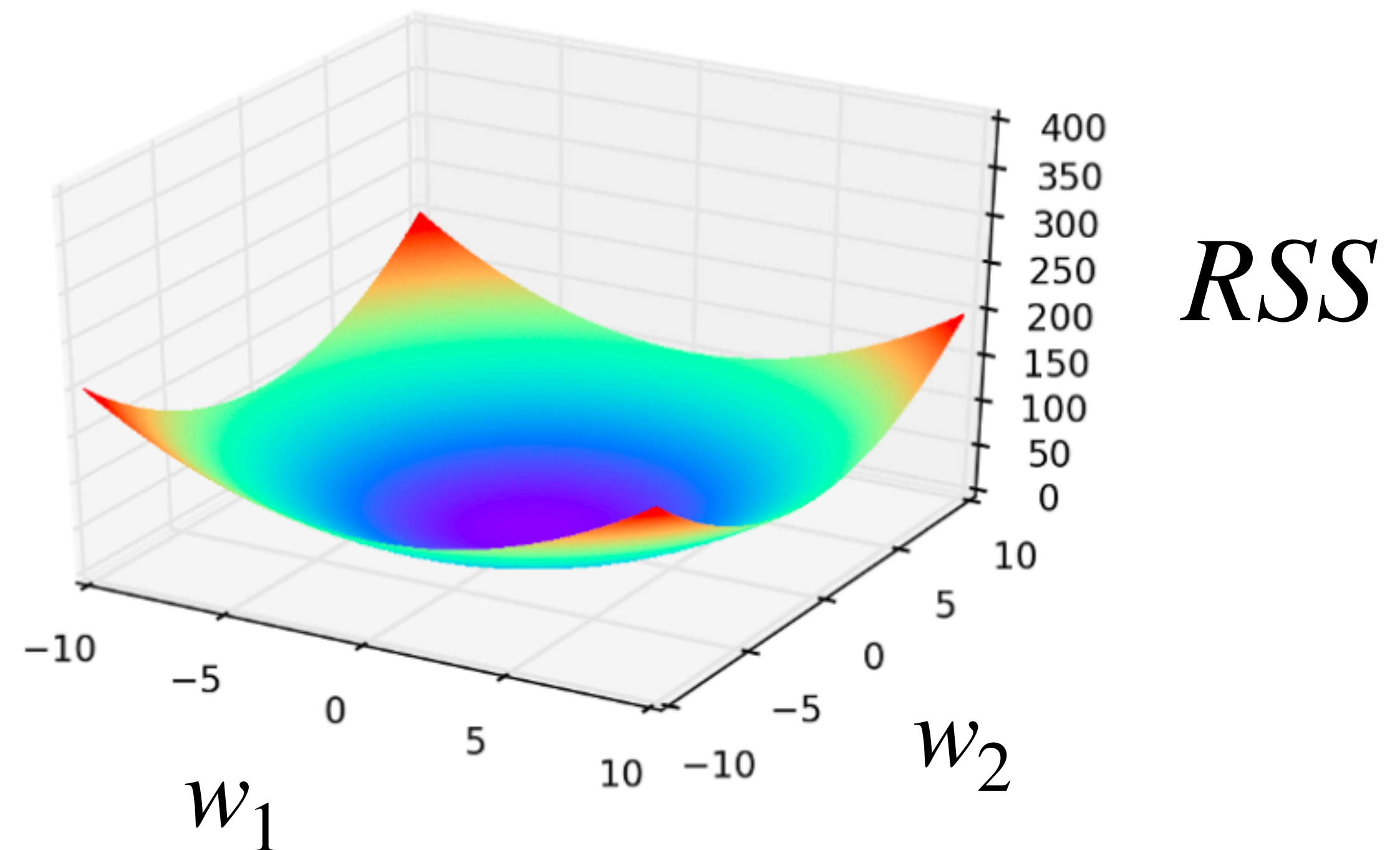
Note: No intercept term in constraints.

RESTRICTING MODEL COMPLEXITY: REGULARIZATION

For better intuition, consider $w_0 = 0$ and 2 coefficients w_1, w_2 :

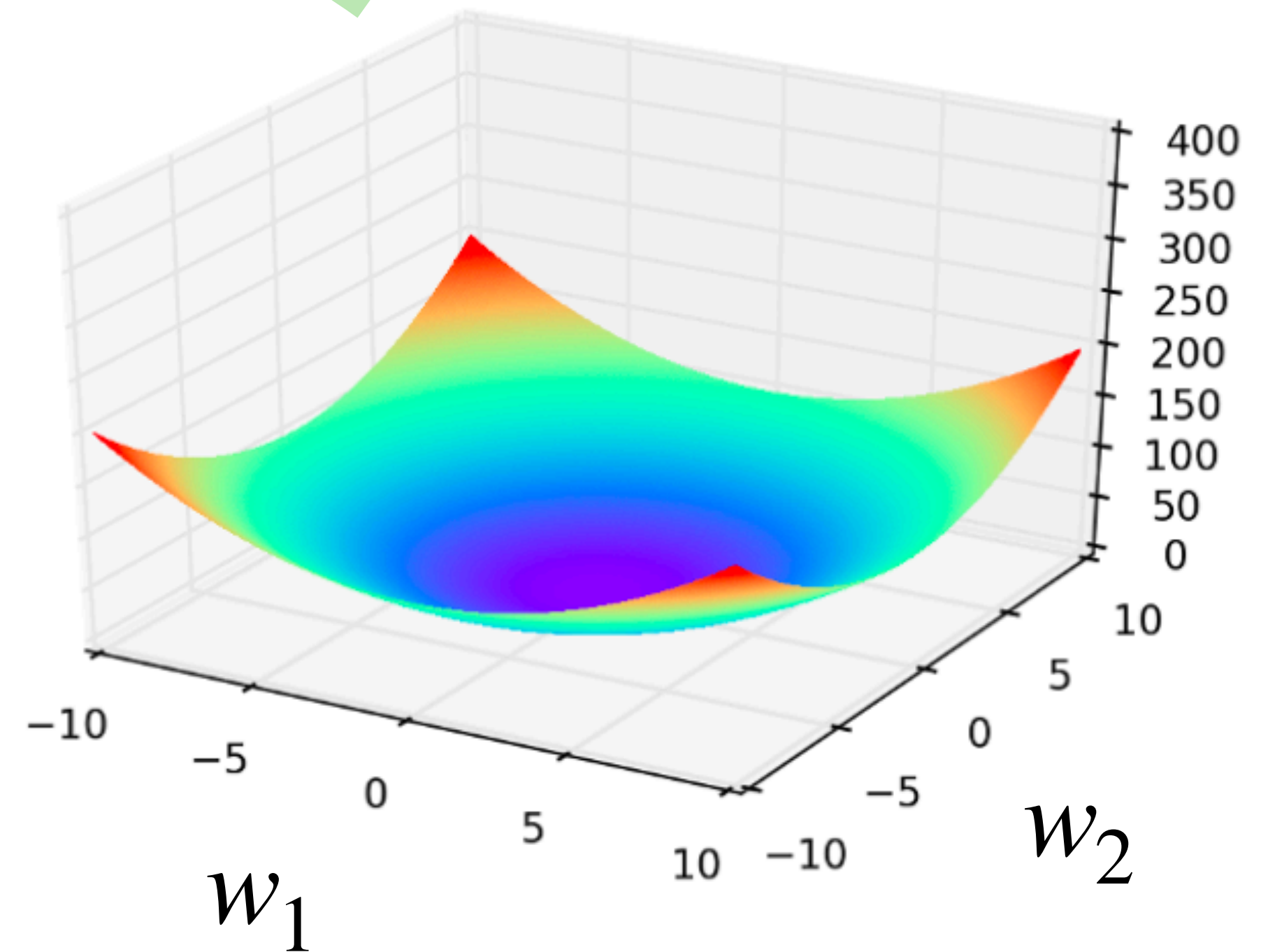
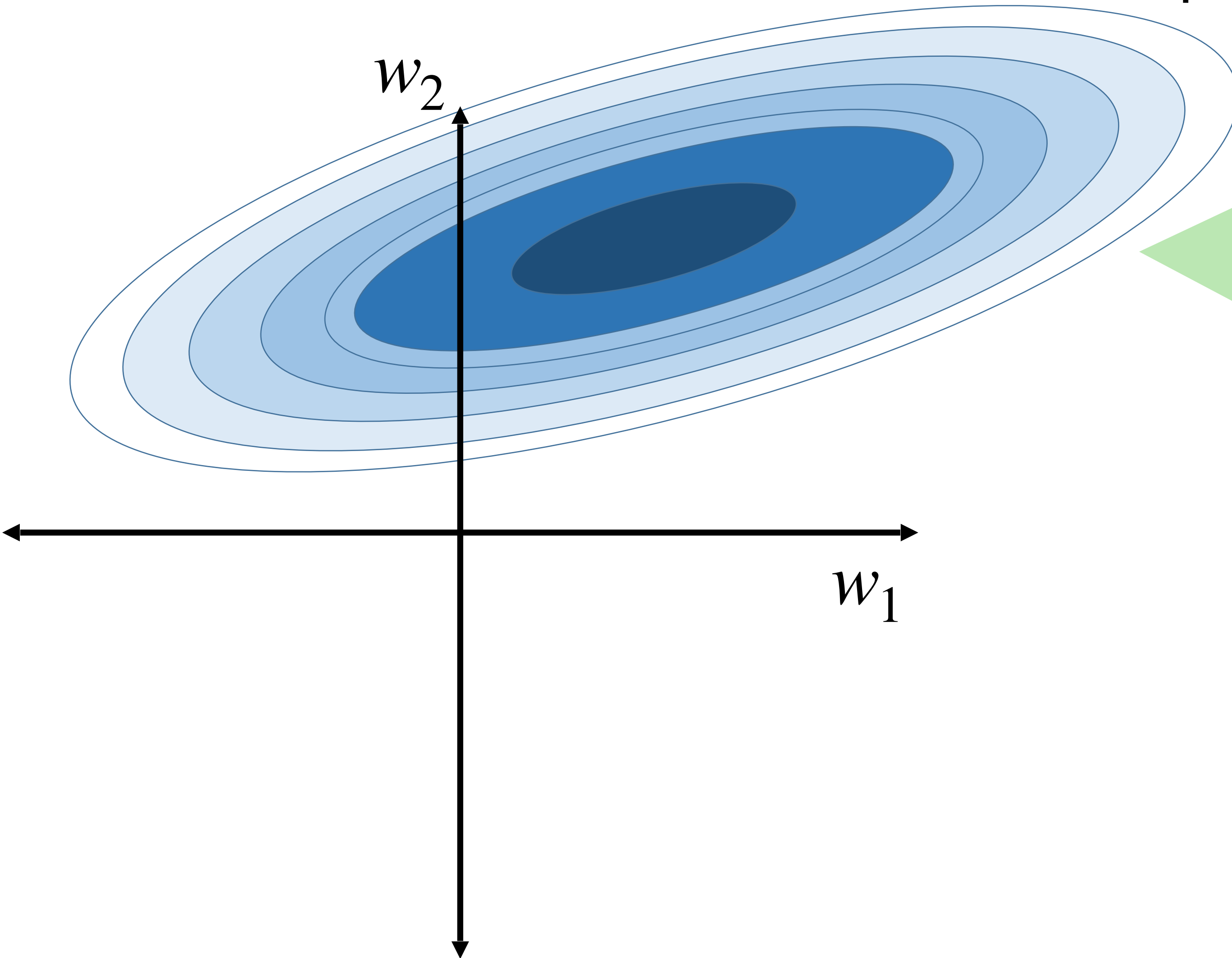
Minimize

$$RSS = \sum_{i=1}^n (y_i - (w_1 x_{i1} + w_2 x_{i2}))^2$$



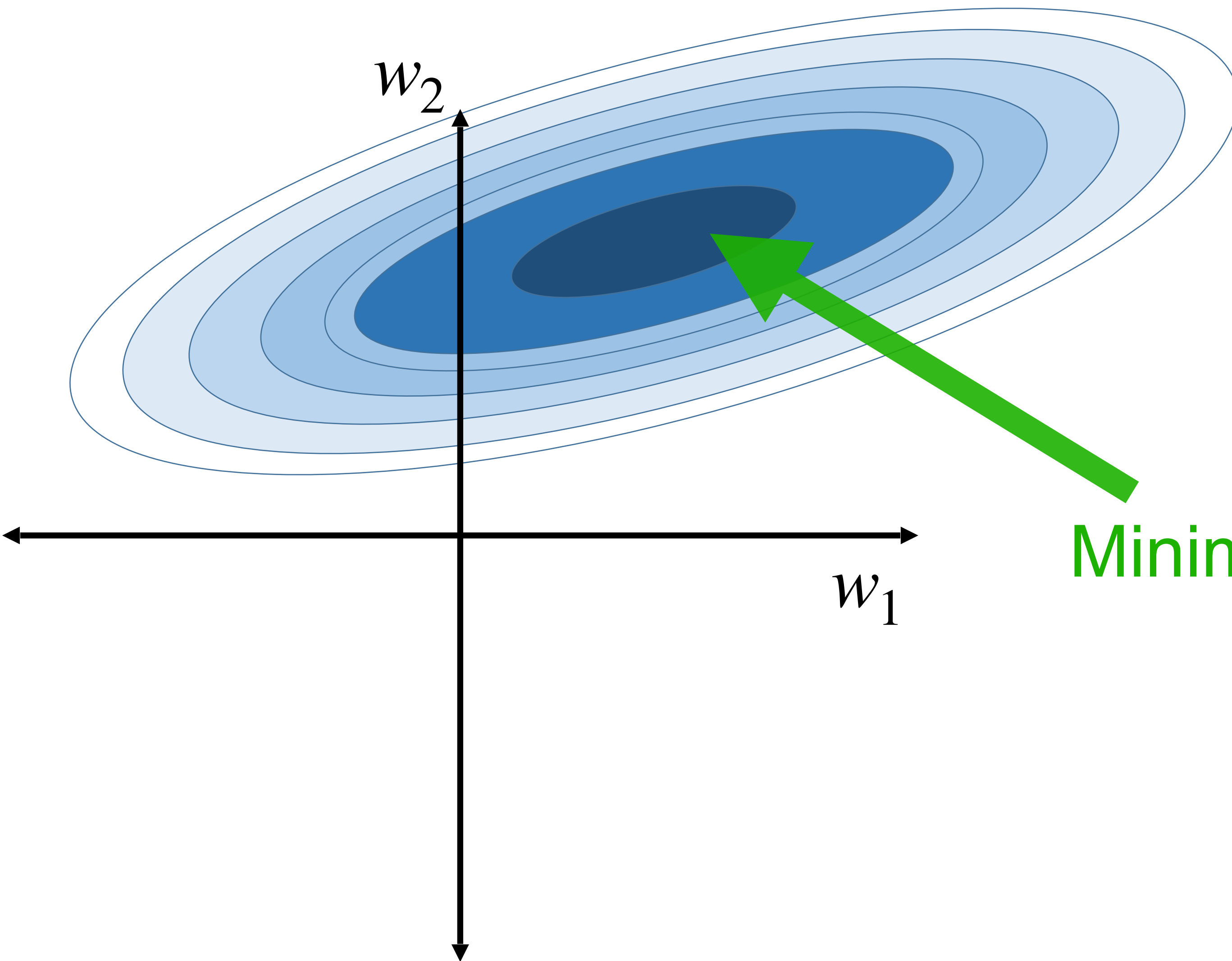
RESTRICTING MODEL COMPLEXITY: REGULARIZATION

Loss surface as contour map



RSS

RESTRICTING MODEL COMPLEXITY: REGULARIZATION

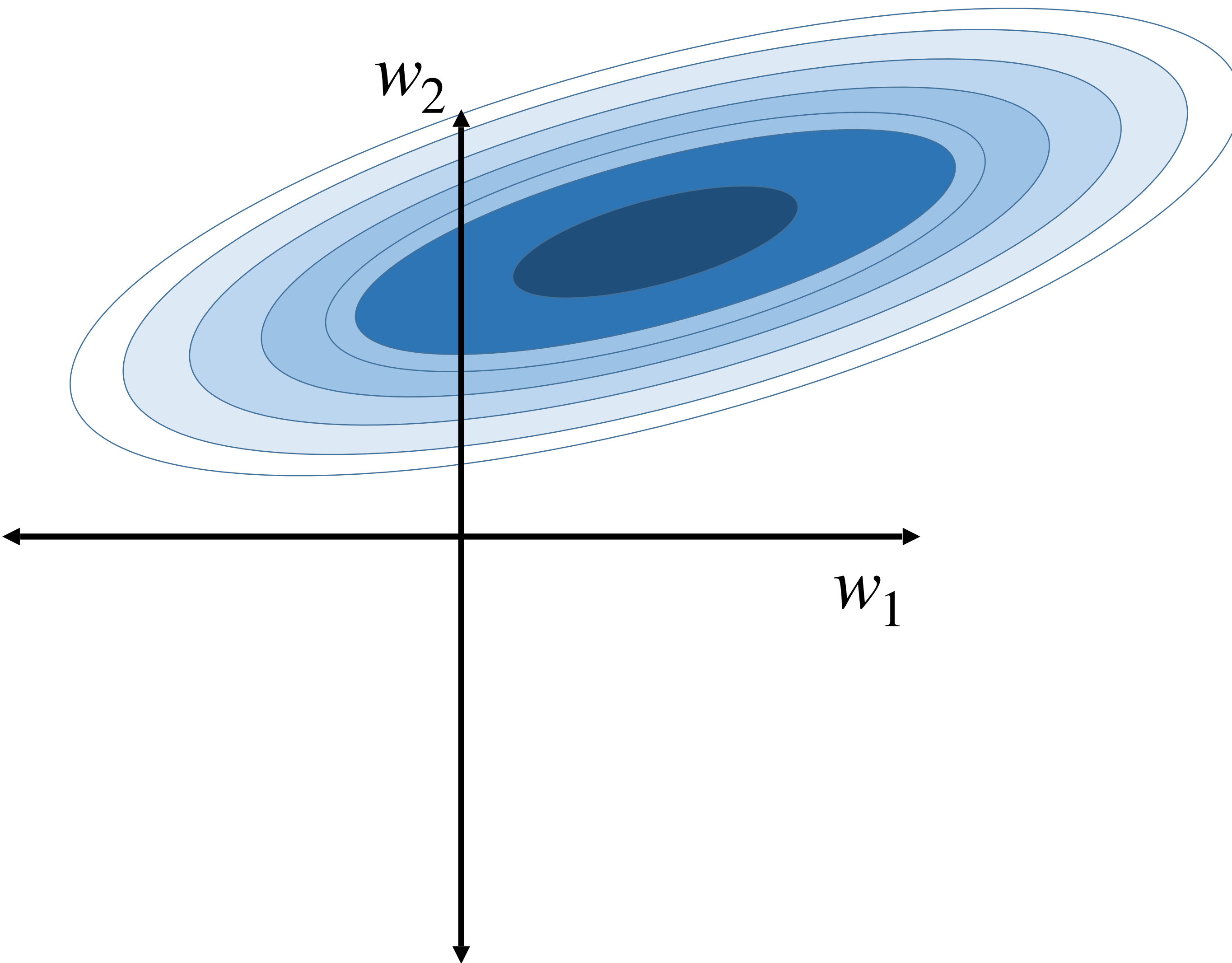


Minimize

$$RSS = \sum_{i=1}^n (y_i - (w_1 x_{i1} + w_2 x_{i2}))^2$$

Minimum loss achieved here.

RESTRICTING MODEL COMPLEXITY: REGULARIZATION



Minimize

$$RSS = \sum_{i=1}^n (y_i - (w_1 x_{i1} + w_2 x_{i2}))^2$$

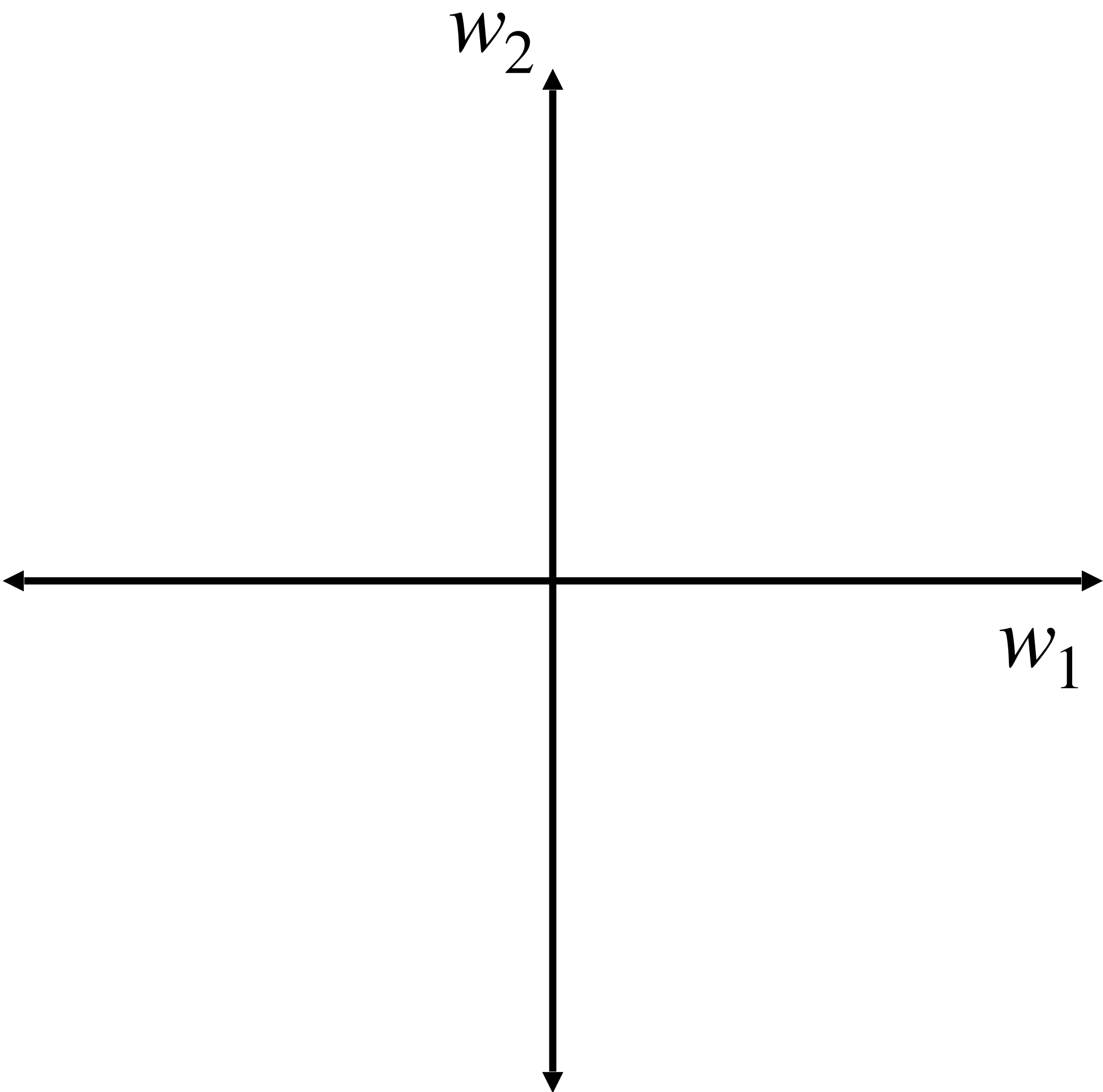
Subject to constraint:

$$\sum_{i=1}^2 |w_i| \leq t$$

L1 REGULARIZATION (LASSO)



RESTRICTING MODEL COMPLEXITY: REGULARIZATION

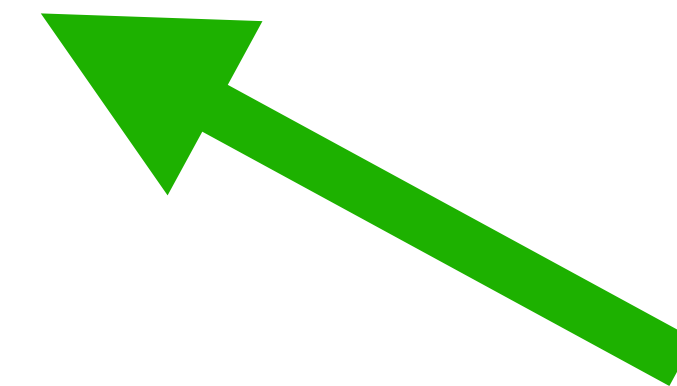


Minimize

$$RSS = \sum_{i=1}^n (y_i - (w_1 x_{i1} + w_2 x_{i2}))^2$$

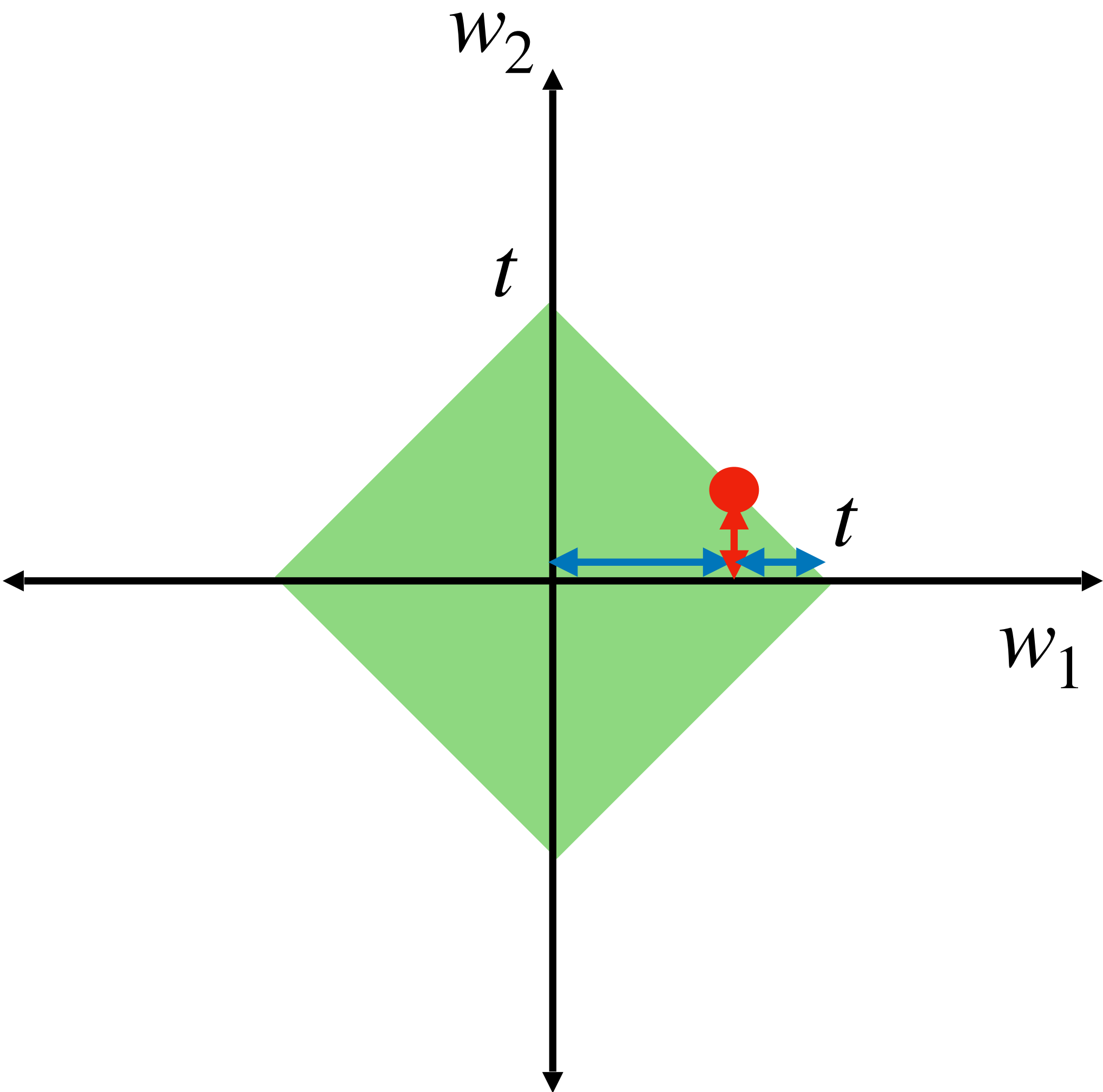
Subject to constraint:

$$\sum_{i=1}^2 |w_i| \leq t$$



What does
the region
satisfying this
look like?

RESTRICTING MODEL COMPLEXITY: REGULARIZATION

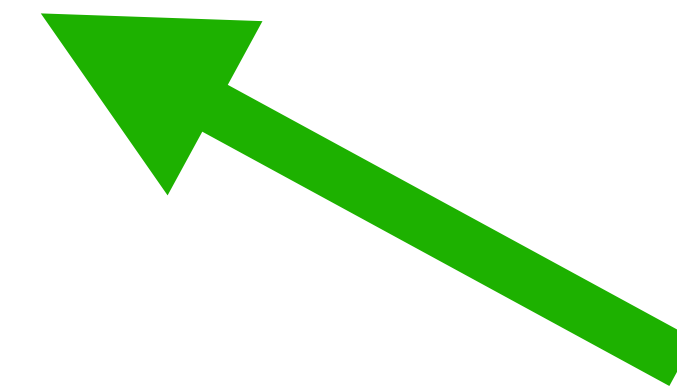


Minimize

$$RSS = \sum_{i=1}^n (y_i - (w_1 x_{i1} + w_2 x_{i2}))^2$$

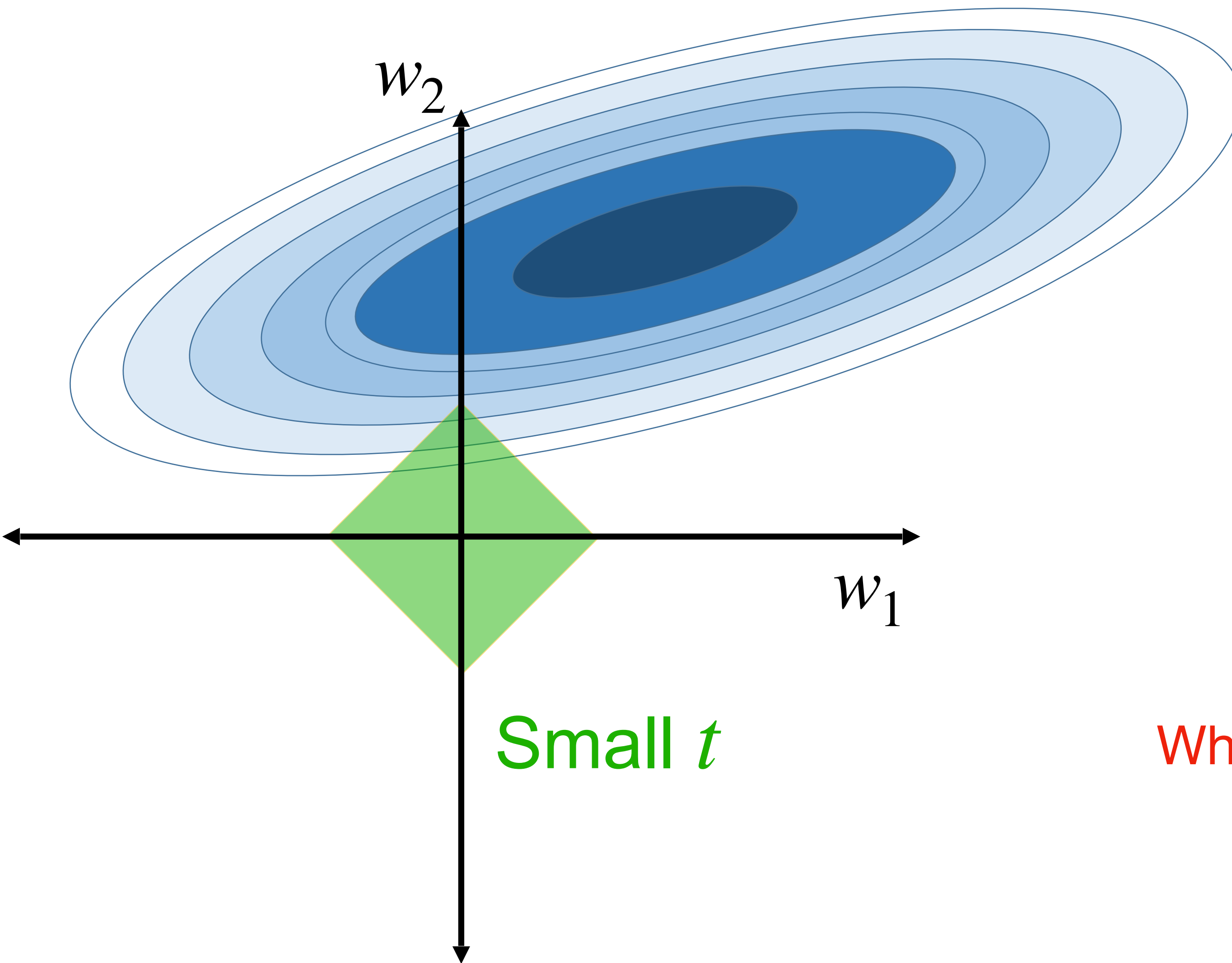
Subject to constraint:

$$\sum_{i=1}^2 |w_i| \leq t$$



Satisfied
anywhere
inside square

RESTRICTING MODEL COMPLEXITY: REGULARIZATION



Minimize

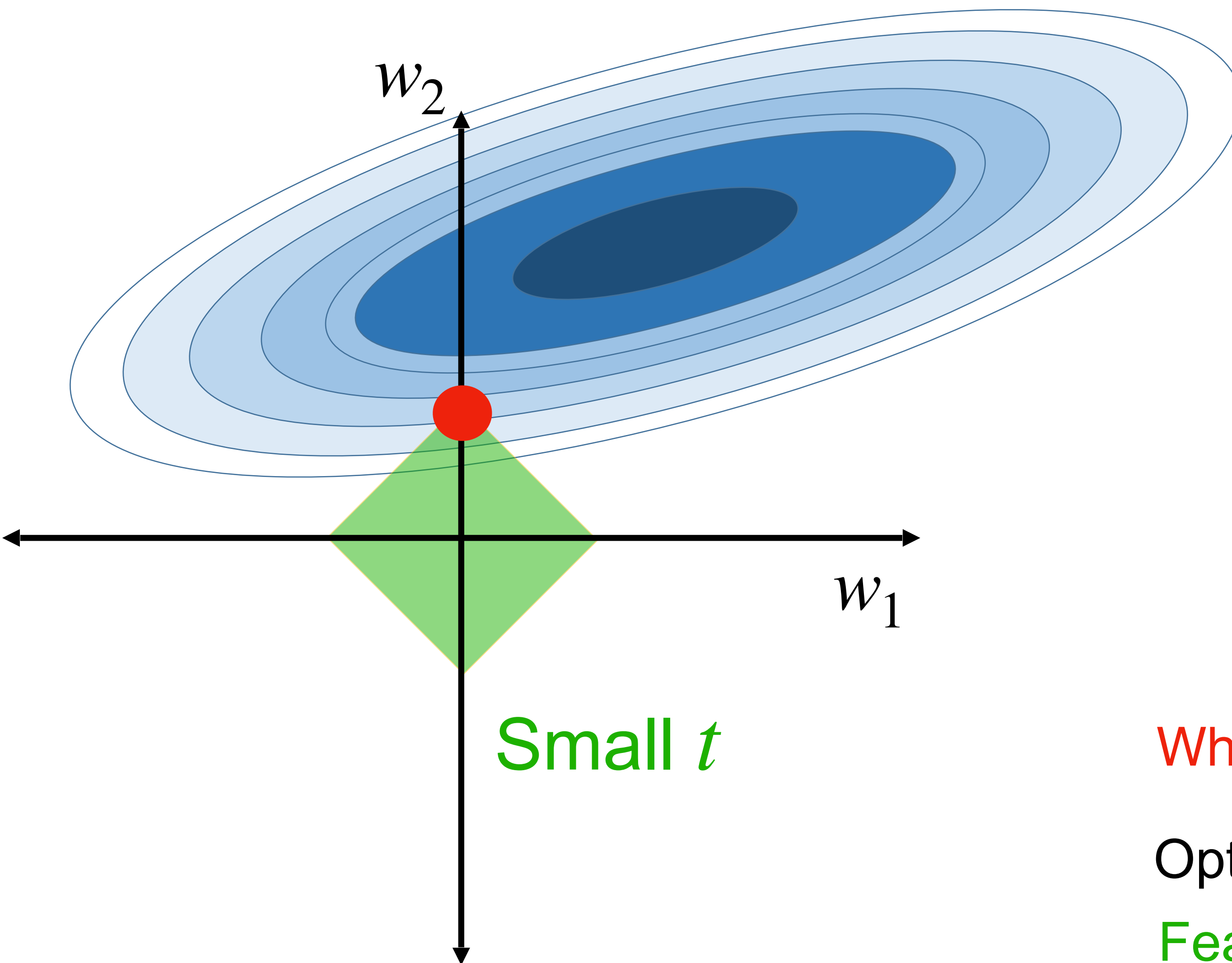
$$RSS = \sum_{i=1}^n (y_i - (w_1 x_{i1} + w_2 x_{i2}))^2$$

Subject to constraint:

$$\sum_{i=1}^2 |w_i| \leq t$$

Where is the optimum coefficient setting?

RESTRICTING MODEL COMPLEXITY: REGULARIZATION



Minimize

$$RSS = \sum_{i=1}^n (y_i - (w_1 x_{i1} + w_2 x_{i2}))^2$$

Subject to constraint:

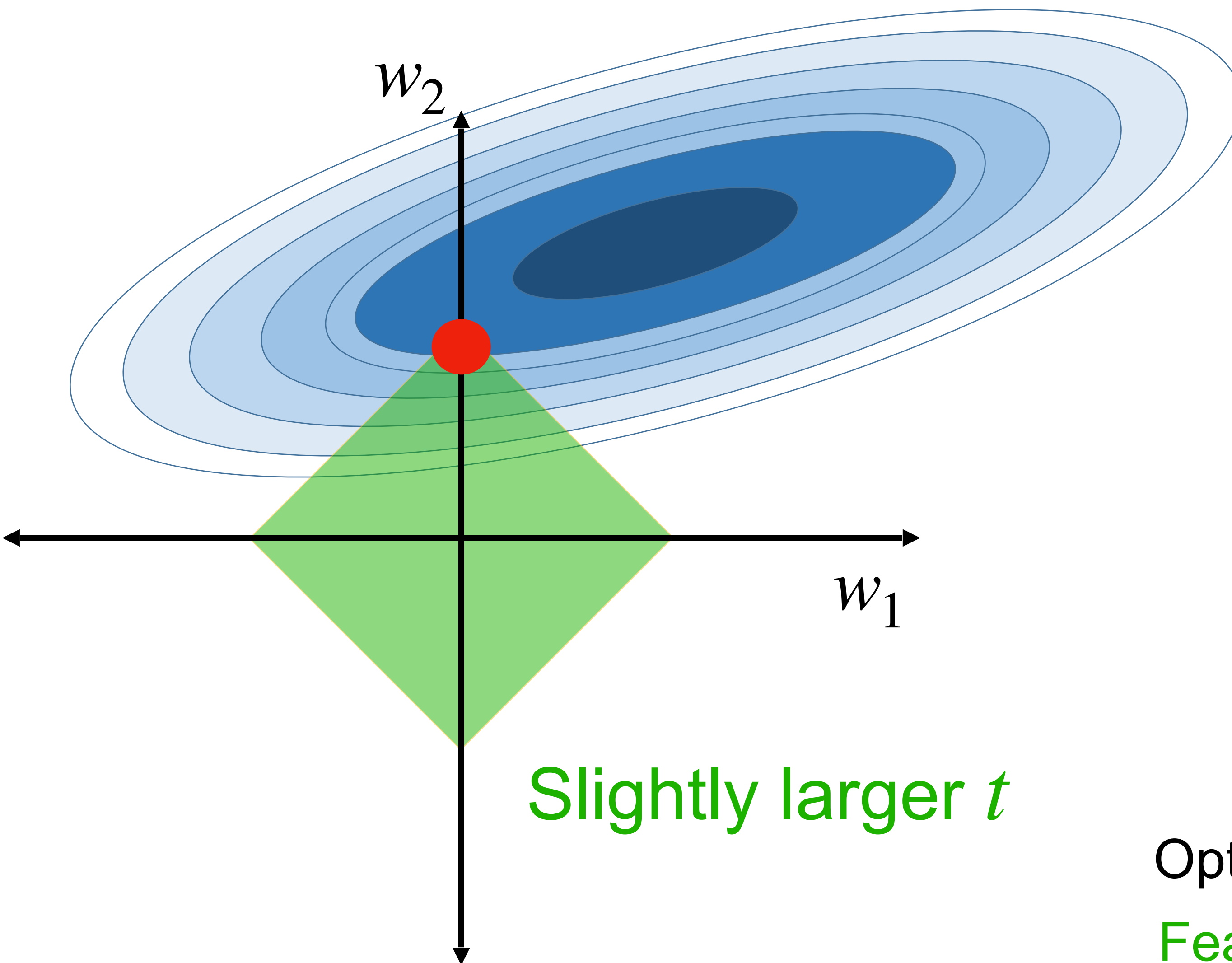
$$\sum_{i=1}^2 |w_i| \leq t$$

Where is the optimum coefficient setting?

Optimum coefficient setting at **Red point**.

Feature w_1 contributes none, model is simple.

RESTRICTING MODEL COMPLEXITY: REGULARIZATION



Minimize

$$RSS = \sum_{i=1}^n (y_i - (w_1 x_{i1} + w_2 x_{i2}))^2$$

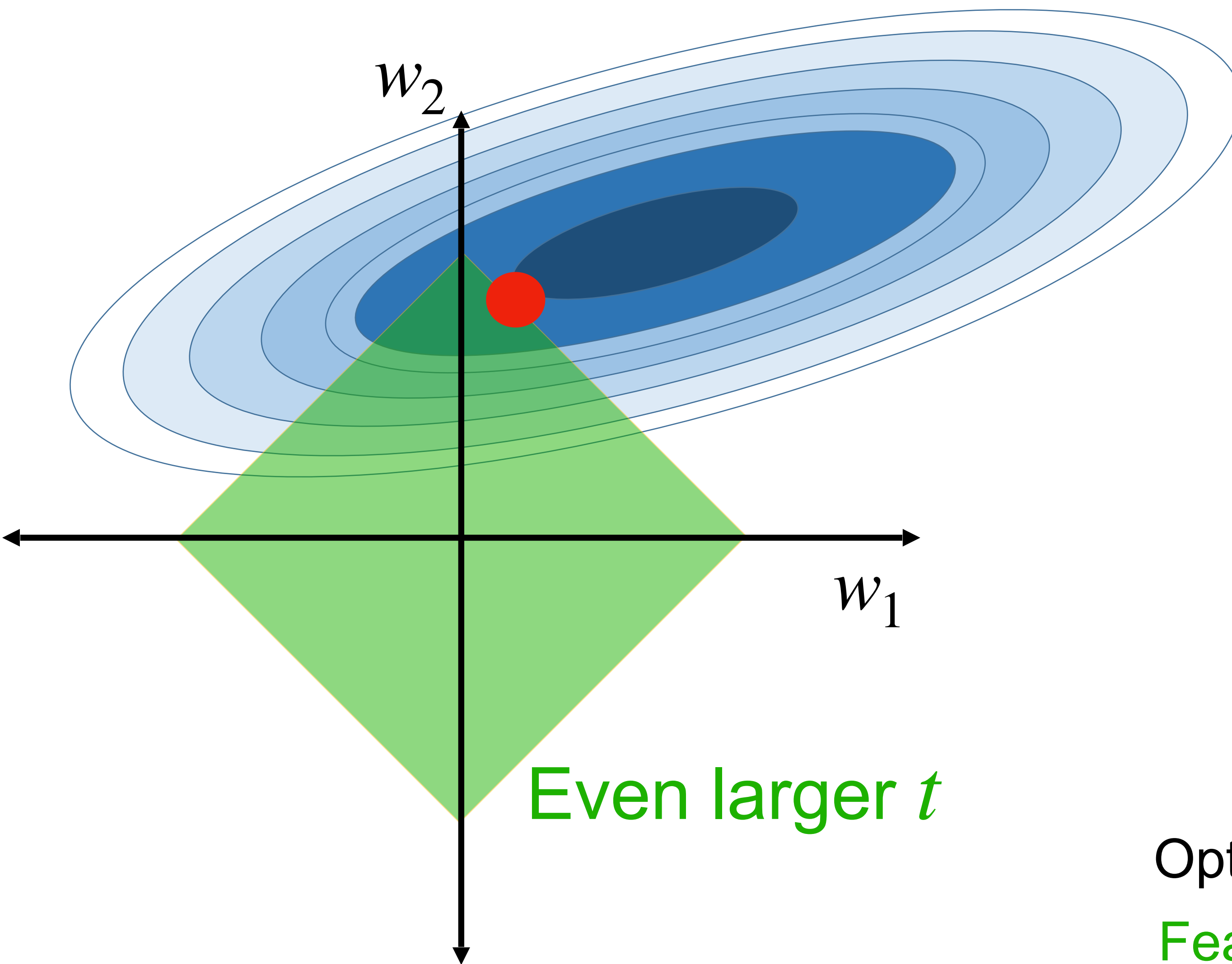
Subject to constraint:

$$\sum_{i=1}^2 |w_i| \leq t$$

Optimum coefficient setting at Red point.

Feature w_1 contributes none, model is simple.

RESTRICTING MODEL COMPLEXITY: REGULARIZATION



Minimize

$$RSS = \sum_{i=1}^n (y_i - (w_1 x_{i1} + w_2 x_{i2}))^2$$

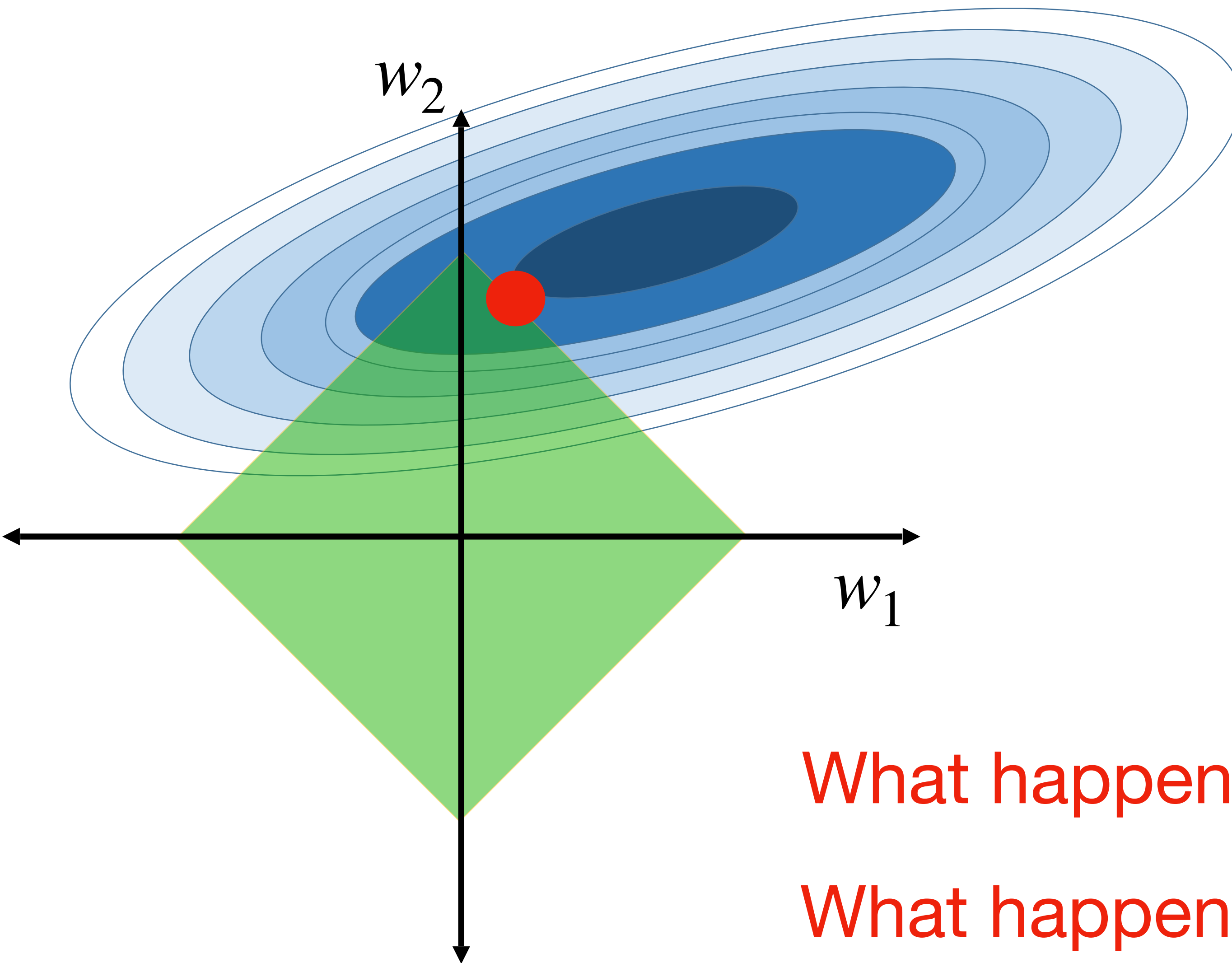
Subject to constraint:

$$\sum_{i=1}^2 |w_i| \leq t$$

Optimum coefficient setting at **Red point**.

Feature w_1 contributes, model more complex.

RESTRICTING MODEL COMPLEXITY: REGULARIZATION



Minimize

$$RSS = \sum_{i=1}^n (y_i - (w_1 x_{i1} + w_2 x_{i2}))^2$$

Subject to constraint:

$$\sum_{i=1}^2 |w_i| \leq t$$

What happens when t very large, say $t \rightarrow \infty$?

What happens when t very small, say $t \approx 0$?

RESTRICTING MODEL COMPLEXITY: REGULARIZATION

Lasso: Optimization Formulation

Minimize $\sum_{i=1}^n (y_i - (w_0 + w_1 x_{i1} + w_2 x_{i2} + \cdots + w_m x_{im}))^2$ **subject to** $\sum_{i=1}^m |w_i| \leq t$

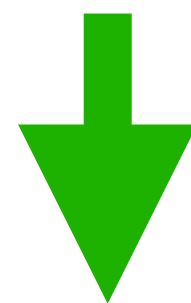
RESTRICTING MODEL COMPLEXITY: REGULARIZATION

Lasso: Optimization Formulation

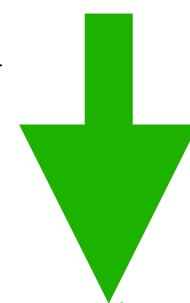
Minimize $\sum_{i=1}^n (y_i - (w_0 + w_1x_{i1} + w_2x_{i2} + \cdots + w_mx_{im}))^2$ subject to $\sum_{i=1}^m |w_i| \leq t$



Minimize $\sum_{i=1}^n (y_i - (w_0 + w_1x_{i1} + w_2x_{i2} + \cdots + w_mx_{im}))^2 + \lambda \sum_{i=1}^m |w_i|$



Keep MSE small



Keep coefficients small.
Simpler model.

LASSO (Least absolute shrinkage and selection operator)

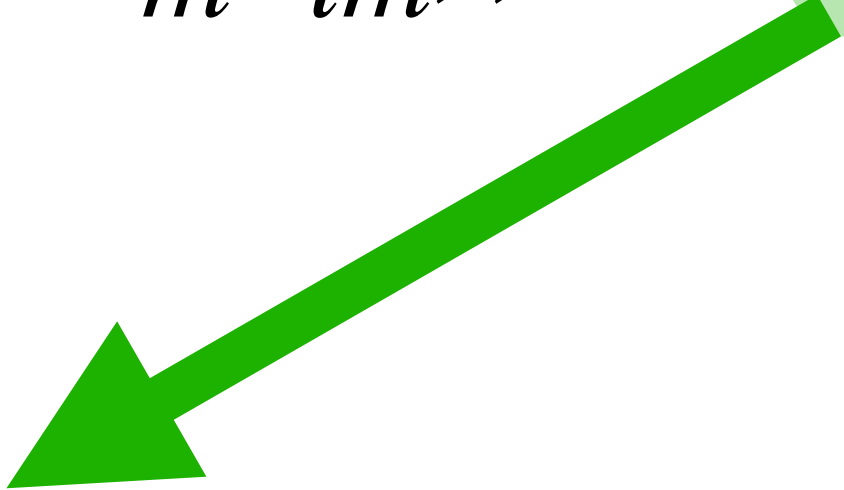
RESTRICTING MODEL COMPLEXITY: REGULARIZATION

Lasso: Optimization Formulation

Minimize $\sum_{i=1}^n (y_i - (w_0 + w_1x_{i1} + w_2x_{i2} + \cdots + w_mx_{im}))^2$ subject to $\sum_{i=1}^m |w_i| \leq t$

$\equiv$

Minimize $\sum_{i=1}^n (y_i - (w_0 + w_1x_{i1} + w_2x_{i2} + \cdots + w_mx_{im}))^2 + \lambda \sum_{i=1}^m |w_i|$



Regularization penalty:
Determine good value using
cross validation

RESTRICTING MODEL COMPLEXITY: REGULARIZATION

Minimize

$$RSS = \sum_{i=1}^n (y_i - (w_1 x_{i1} + w_2 x_{i2}))^2$$

Subject to constraint:

$$\sum_{i=1}^m w_i^2 \leq t$$

L2 REGULARIZATION (RIDGE)

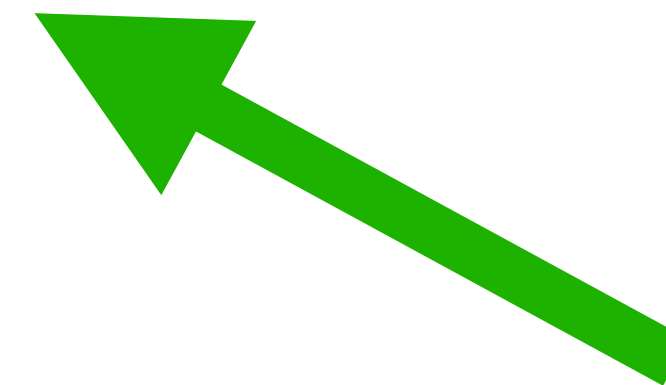
RESTRICTING MODEL COMPLEXITY: REGULARIZATION

Minimize

$$RSS = \sum_{i=1}^n (y_i - (w_1 x_{i1} + w_2 x_{i2}))^2$$

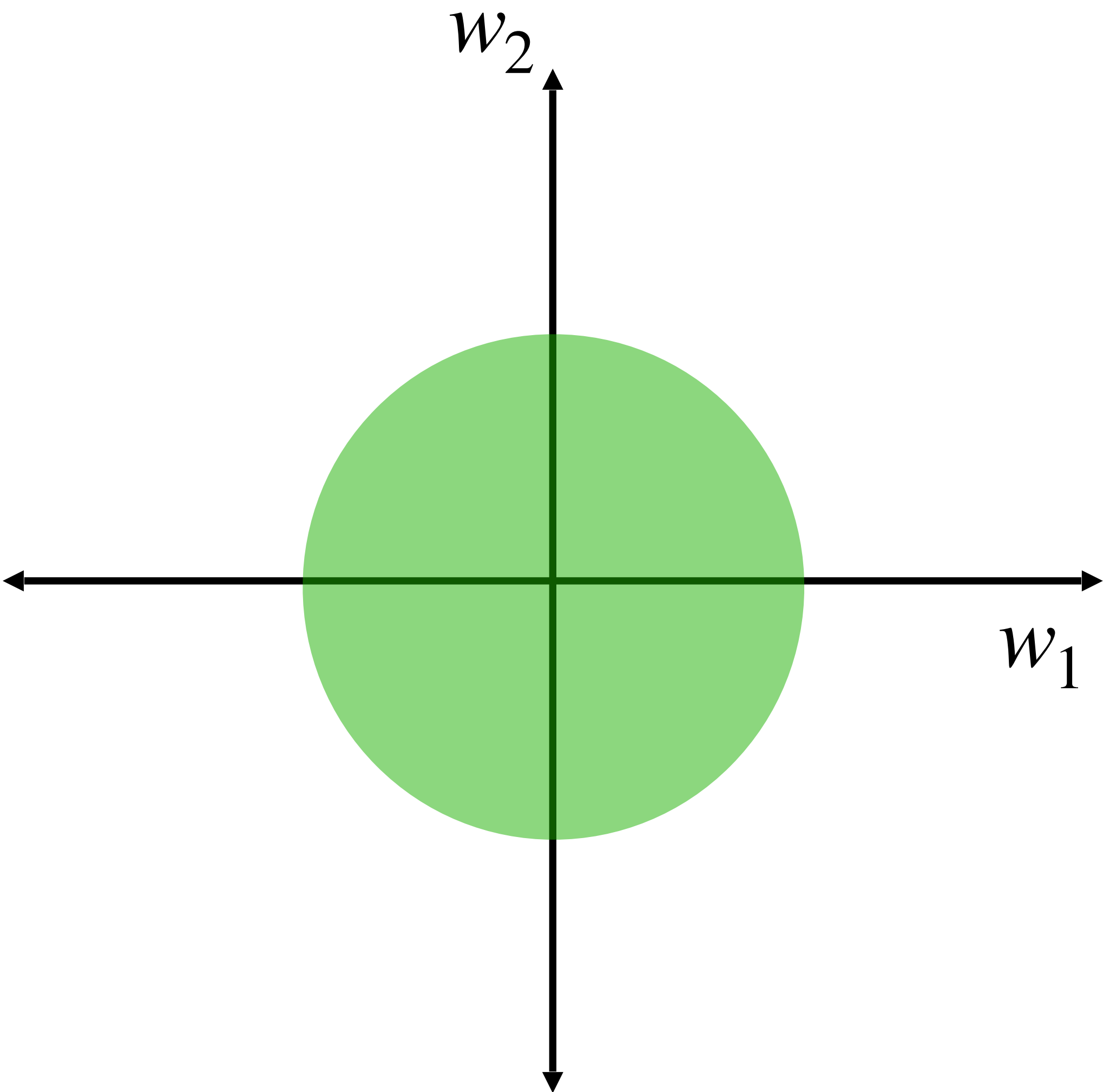
Subject to constraint:

$$\sum_{i=1}^m w_i^2 \leq t$$



What does
the region
satisfying this
look like?

RESTRICTING MODEL COMPLEXITY: REGULARIZATION

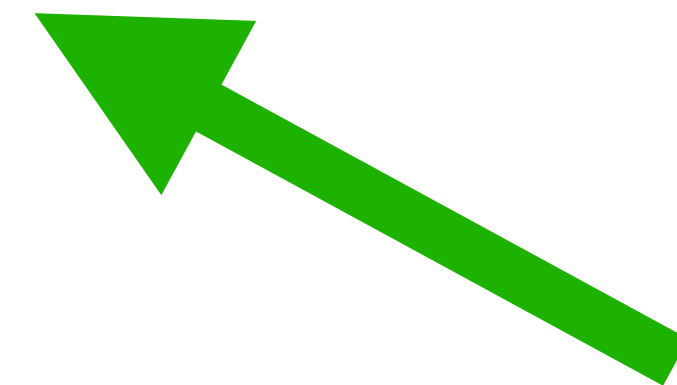


Minimize

$$RSS = \sum_{i=1}^n (y_i - (w_1 x_{i1} + w_2 x_{i2}))^2$$

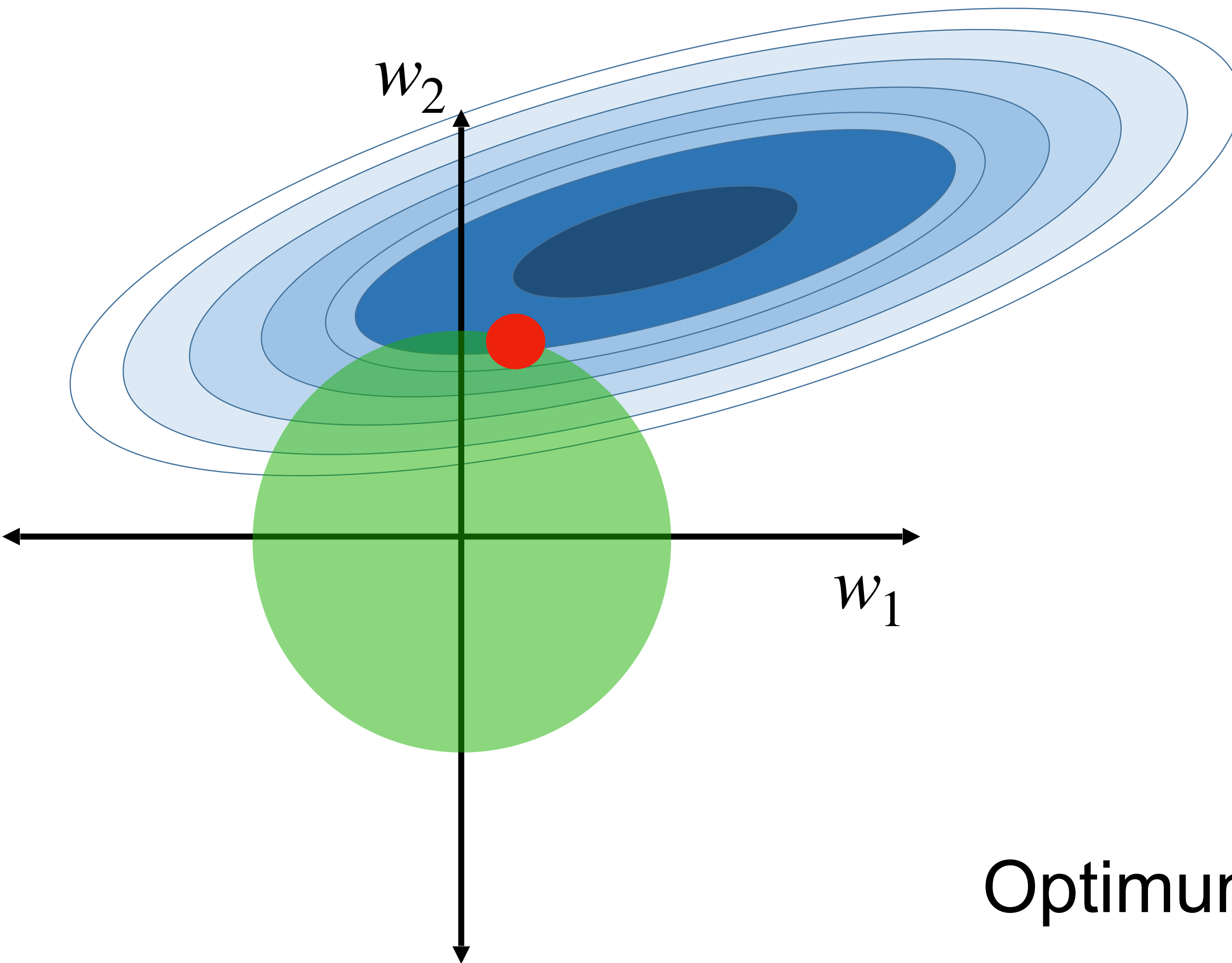
Subject to constraint:

$$\sum_{i=1}^m w_i^2 \leq t$$



What does
the region
satisfying this
look like?

RESTRICTING MODEL COMPLEXITY: REGULARIZATION



Minimize

$$RSS = \sum_{i=1}^n (y_i - (w_1 x_{i1} + w_2 x_{i2}))^2$$

Subject to constraint:

$$\sum_{i=1}^m w_i^2 \leq t$$

Optimum coefficient setting at **Red point**.

RESTRICTING MODEL COMPLEXITY: REGULARIZATION

Ridge: Optimization Formulation

$$\text{Minimize } \sum_{i=1}^n (y_i - (w_0 + w_1 x_{i1} + w_2 x_{i2} + \cdots + w_m x_{im}))^2 \quad \text{subject to } \sum_{i=1}^m w_i^2 \leq t$$

$\equiv$

$$\text{Minimize } \sum_{i=1}^n (y_i - (w_0 + w_1 x_{i1} + w_2 x_{i2} + \cdots + w_m x_{im}))^2 + \lambda \sum_{i=1}^m w_i^2$$

RESTRICTING MODEL COMPLEXITY: REGULARIZATION

Ex: Hitters data set.

...	Years	CAtBat	CHits	CHmRun	CRuns	CRBI	CWalks	League	Division	PutOuts	Assists	Errors	Salary
	1	293	66	1	30	29	14	A	E	446	33	20	NaN
	14	3449	835	69	321	414	375	N	W	632	43	10	475.0
	3	1624	457	63	224	266	263	A	W	880	82	14	480.0
	11	5628	1575	225	828	838	354	N	E	200	11	3	500.0
	2	396	101	12	48	46	33	N	E	805	40	4	91.5

RESTRICTING MODEL COMPLEXITY: REGULARIZATION

Scale the features

```
# in regularization it is important to scale features
from sklearn.preprocessing import StandardScaler

# we do scaling after train/test split to avoid data leakage
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, train_size=0.8, random_state=0)

scaler = StandardScaler().fit(X_train)
X_train[X.columns] = scaler.transform(X_train[X.columns])
X_test[X.columns] = scaler.transform(X_test[X.columns])
```

Years	CAtBat	CHits	CHmRun	CRuns	CRBI	CWalks	PutOuts	Assists	Errors
-0.464631	-1.001763	-0.980581	-0.897367	-0.957761	-0.945831	-0.884467	-0.613848	0.967475	0.443495
-0.249713	0.279628	0.353806	0.995644	0.567252	0.535436	0.249045	-0.189638	2.410126	0.592988
1.469625	1.857587	2.086141	2.618225	1.970388	2.626043	0.967858	-0.058581	-0.793506	-0.004983
-0.894465	-0.894980	-0.851090	-0.498839	-0.814009	-0.683242	-0.706738	-0.475894	-0.807052	-0.154476
3.618797	5.144516	5.610819	1.379940	5.679792	3.370043	5.197794	0.786390	-0.563224	-0.453462

Scaling after train/
test split to avoid
data leakage

RESTRICTING MODEL COMPLEXITY: REGULARIZATION

Apply Lasso

```
# apply lasso with arbitrarily set alpha=1
from sklearn.linear_model import Lasso

reg = Lasso(alpha=1)
reg.fit(X_train, y_train)
reg.score(X_train, y_train)
```

alpha here is our λ .

0.49169521567171304

```
# find mse on the train and test data
predict_train = reg.predict(X_train)
mse_train = mean_squared_error(y_train, predict_train)
print('training MSE : ', mse_train)

predict_test = reg.predict(X_test)
mse_test = mean_squared_error(y_test, predict_test)
print('test MSE : ', mse_test)
```

training MSE : 88524.3983554207
test MSE : 117768.90478161338

RESTRICTING MODEL COMPLEXITY: REGULARIZATION

Try different alpha values and plot mse values.

```
alphas = np.linspace(0.01,300,100)
coefs = []
for alpha in alphas:
    lasso_reg = Lasso(alpha=alpha)
    lasso_reg.fit(X_train, y_train)
    coefs.append(lasso_reg.coef_)
```

For the homework log space

